MYAVANA Technical Documentation

Table of Contents

Introduction....5
Process....6
Requirements Definition....6
Development....7
Testing....7
Release....7
Credentials....8
Products....9
MYAVANA AI Service....9
Source Code Information....9
Deployment Information....10
Staging Environment....10
Production Environment....10
Usage/Integration Documentation....11
MYAVANA Retail Solution....14
Setting Up New Retail Partners....15
HairAI Only Integration:....15
Retail Front-end with Product Integration:....18
High Level Architecture....22
Database Design....24
Product and related attribute definitions....24
Tenant and related attribute definitions....25
User management....25
Current Recommendation Logic....26
High Level Diagram....26
Recommendation Logic Flow....27
Explanation of Algorithm....28
Source Code and Design Information....29
Front-end Source Information....29
Retail Portal Service Source Information....29
Admin Portal Source Information....30
Deployment Information....31
Staging Environment Deployment....31
Production Environment Deployment....31
MYAVANA Solution....33
Admin Portal Overview....33
High Level Architecture....36
1. Frontend and API Gateway....36

2. Microservices..... 36
3. Utilities..... 36
4. Data Layer..... 37
5. Additional Component: MyAvana.ExcelImport..... 37
Connection Between Components..... 37
Database Design..... 38
Hair Profile..... 38
Product Recommendation Logic and Auto HHCP Logic..... 42
Auto Generate HHCP..... 42
AutoRecommend Videos..... 43
3. sp_GetStyleRecipeProducts..... 44
Input Parameters:..... 44
Logic Breakdown:..... 44
1. Retrieve Latest Questionnaire Data:..... 44
2. Temporary Tables Creation:..... 44
3. Mapping User Selections to Temporary Tables:..... 45
4. Create Product-Product Type Relationship:..... 45
5. Identify Matching Products:..... 45
6. Rank and Filter Products:..... 45
7. Return Top 3 Products:..... 45
8. Output:..... 46
9. Temporary Tables Cleanup:..... 46
Source Code and Design Information..... 46
Admin Portal Source Information..... 46
Myavana Backend Service Source Information..... 46
Deployment Information..... 47
Staging Environment Deployment..... 47
Production Environment Deployment..... 48
Customer Portal Overview..... 49
High Level Architecture..... 51
Database Design..... 51
Product Recommendation Logic and Auto HHCP Logic..... 52
Source Code and Design Information..... 52
Customer Portal Source Information..... 52
Deployment Information..... 52
Staging Environment Deployment..... 52
Salon Portal..... 54
Architectural Overview..... 55
MYAVANA Pro Solution..... 55
Admin Portal Pro Overview..... 55
Myavana Pro High-Level Architecture..... 56

Target Framework....56
Key Components....57
Mobile Applications....60
MYAVANA AI....60
UI Design (iOS/Android)....60
Purpose of the Registration Page....62
Backend Logic of the Registration Page....62
Build App in Xcode....71
TestFlight: Upload and Manage Staging/Production Environments....71
Releasing to the App Store....72
Android App....73
Handling Staging and Production Environments....73
App Signing Configuration....74
Building/Generate APKs for Staging and Production....76
Distributing APKs....77
Uploading to Google Play (Production)....77
MYAVANA PRO....81
Login Page....81
Analytics Tab....82
Other Available Tabs:....82
Hair Analysis Button....83
Re-run Hair AI Tool....83
Android App....88
Third Party Tools....90
AWS Infrastructure Documentation....93
Servers....93
Databases....93
General How To’s....95

Introduction

This document provides in-depth technical details on the software and infrastructural assets that power the MYAVANA suite of offerings. Where applicable it provides the software architectural details, inter-relationships amongst the services and how to deploy them to their hosting environment.

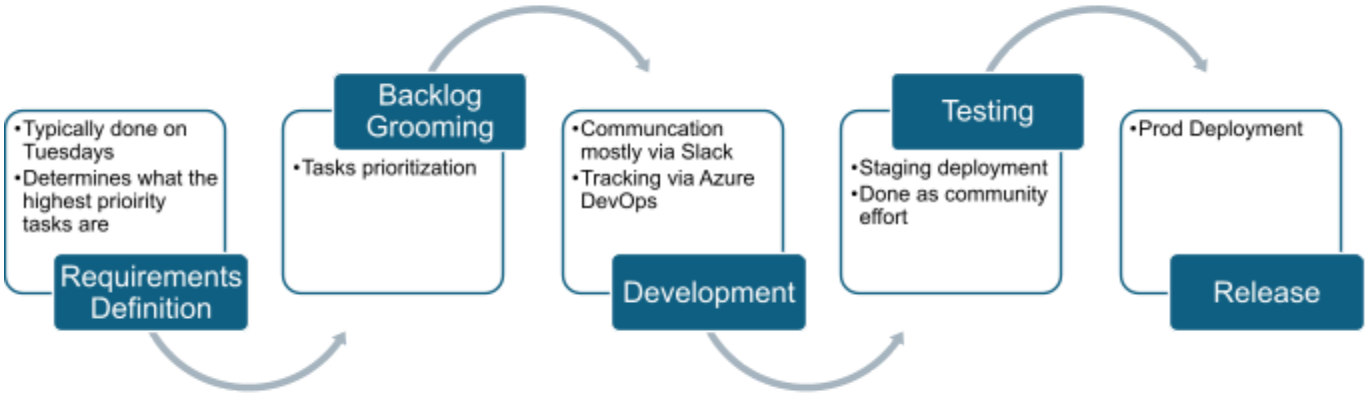
This document is organized by Products/Software Services.

Tech Stack

Technology	Description
.NET 3.x and 6.x	
AWS	
ASP.NET Core Razor	
JavaScript	
ASP.NET Core 2.2	
ASP.NET Core 6.0	
SQL Server	
Microservices Architecture	
Clean Architecture Model	
React Native (IOS/Android)	
HTML	

Process

Given that we are still a start-up, our process is focused on agility and being able to pivot at short notice when opportunities present themselves. We therefore don't run our sprints on a well-defined cadence. The current development process is as illustrated below:



Requirements Definition

This is primarily done at the Weekly Tuesday Tech Call meeting (1PM EST). All stakeholders (Leadership, R & D, Marketing, Data Science etc.) participate in this call and bring up features, improvements, bugs etc. that need to be addressed. We also use this call to communicate recent releases and what's in the pipeline. Occasionally urgent requirements are discussed in Slack particularly on the #technology, #mobiledev and #webapp channels.

We keep a running meeting notes of our discussion on this google doc: https://docs.google.com/document/d/1Q3e9zKgiJ7ZoOpr39ZSZeWFpJ1LQ_CcDVHVUNAOhhzc/edit?tab=t.0

Backlog Grooming

Our development activity is managed using a combination of Azure DevOps and GitHub. Project management is via Azure DevOps while GitHub is our source code repository tool.

Our backlog is managed as a Kanban board which can be accessed at: https://dev.azure.com/myavanatechnology/General%20Road%20Map/_boards/board/t/General%20Road%20Map%20Team/Issues. We use a four-lane swim lane namely: To Do, Doing, Testing and Done.

Link to DevOps:
https://dev.azure.com/myavanatechnology/General%20Road%20Map/_boards/board/t/General%20Road%20Map%20Team/Issues

Link to GitHub repositories:
https://github.com/MyavanaTechnology

Development

The CTO communicates directly with the dev team on priorities, status updates and actively unblocks any blockers. The communication occurs twice daily: late night and early mornings due to time zone difference. These communications take place on Slack either in private group channels or product channels.

Testing

Once development work is complete, code artifact is pushed out to the staging environment as applicable (i.e. **TestFlight** for iOS app, **APK** build for Android App and **staging environment** for web apps)

Release

Once Tests have passed and the dev team receives the green light, code changes are pushed to the production environment.

Credentials

Our credentials for the infrastructure access and all other third party tools are vaulted in 1Password (https://team-myavana.1password.com/home)

Products

MYAVANA AI Service

The MYAVANA AI Service holds the proprietary Convoluted Neural Network (CNN) that is responsible for classifying images for type and texture. **It takes as input a single image and returns a structured JSON file with details on the Type and Texture of the Image** (API details provided later in this document).

The Api endpoint is protected by a Header Key (x-api-key) which is required to successfully execute the image analysis. The list of approved keys is stored in the appSettings.json file on the prod server. When a new partner is onboarded, a corresponding api key for that customer should be added to the appSettings file (in future we can decide to move the list of keys to a database). ki;9o

Source Code Information

There are two source code versions for the AI service. The application logic in both **is** the same but they are based on different versions of .NET Core.

1. **.NET Core 3.X version**
    a. Source control location: https://github.com/MyavanaTechnology/myavana_classification_ai/tree/TwoAIModel (Current Branch: TwoAIModel)
    b. This is the version used for the ULTA and Unilever integration
    c. An AMI instance to create a server with the version deployed is available in AWS at: https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ImageDetails:imageId=ami-0ecec2360218bbaa9
    d. .NET Core 3.X version is EOL and the original development team didn't want to update versions hence why we still have this running.
2. **.NET Core 6.X version**
    a. Source control location: https://github.com/MyavanaTechnology/HairAnalysis-BE (Current Branch: master)
    b. This is the version used for all our internal applications (Mobile Apps, Retail Kiosk, Salon Portal, Customer Portal)
    c. An AMI instance to create a server with the version deployed is available in AWS at:

https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ImageDetails:imageId=ami-0668d3c903725a1af

d. As at the time of this documentation .NET Core 6.X version is EOL it would be ideal to update the version of the app to a supported version (i.e. .NET Core 8.0)

Deployment Information

Staging Environment

The HairAI service has no staging environment. This was done for cost-saving purposes as this service is compute intensive. All usage points to the production environment.

Production Environment

The Prod HairAI service runs on a single beefy linux machine with the following attributes:

- **Name**: SharedAnalysisAI_01
- **AWS Location**: https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-0e628ae73bc130ee1
- **IP Address**: 52.87.64.1
- **CNAME**: analysisai01.myavana.com
- **Server PATH**: /var/www/analysis
- **Commands**:
  - **Start Service**: sudo systemctl start analysis-api.service
  - **Stop Service**: sudo systemctl stop analysis-api.service
  - **Restart Service**: sudo systemctl restart analysis-api.service
- **Site Definition File**: /etc/nginx/sites-enabled/analysis
- **Service Definition File**: /etc/systemd/system/analysis-api.service

Usage/Integration Documentation

Our HairAI service is used for classifying the type and texture of an input image. Also available at
https://docs.google.com/document/d/1PXcDBqTPnJUscMdGdcvenVFXWafR5gPK/edit

Input Endpoint

It's a simple POST endpoint which takes in a single input parameter (the image). The service endpoint is protected by an Api Key which must be included in the header of every request.

Sample Input

POST /api/v2/Image/classifyimage HTTP/1.1
api-key: <PROVIDED_BY_MYAVANA>
*Accept: */**
Cache-Control: no-cache
Host: <PROVIDED_BY_MYAVANA>
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: multipart/form-data; boundary=--------------------------848554631592511105189015
Content-Length: 41252

--------------------------848554631592511105189015
Content-Disposition: form-data; name="imageFile"; filename="straight.jpg"
<straight.jpg>
--------------------------848554631592511105189015—

API Response

The classification endpoint returns a JSON formatted payload which provides description for the **Type** and **Texture** detected in the input image as well as the percentage weight in each Category. The response is an array with two elements describing the Type and Texture results.

The structure is

Identifier: indicates if the section describes the type or texture result

labels: an array/collection that lists all possible classes for the result detected for identifier class along with their percentage accuracy (see sample payload).

label: indicates the predominant identifier listed in the labels. i.e. the primary Hair Type or Texture

explainer: a brief description of the predominant identifier.

The four possible values of Type returned are

- Straight
- Coily
- Curly
- Wavy

The three possible values of Texture returned are:

- Fine
- Medium
- Coarse

Sample Response

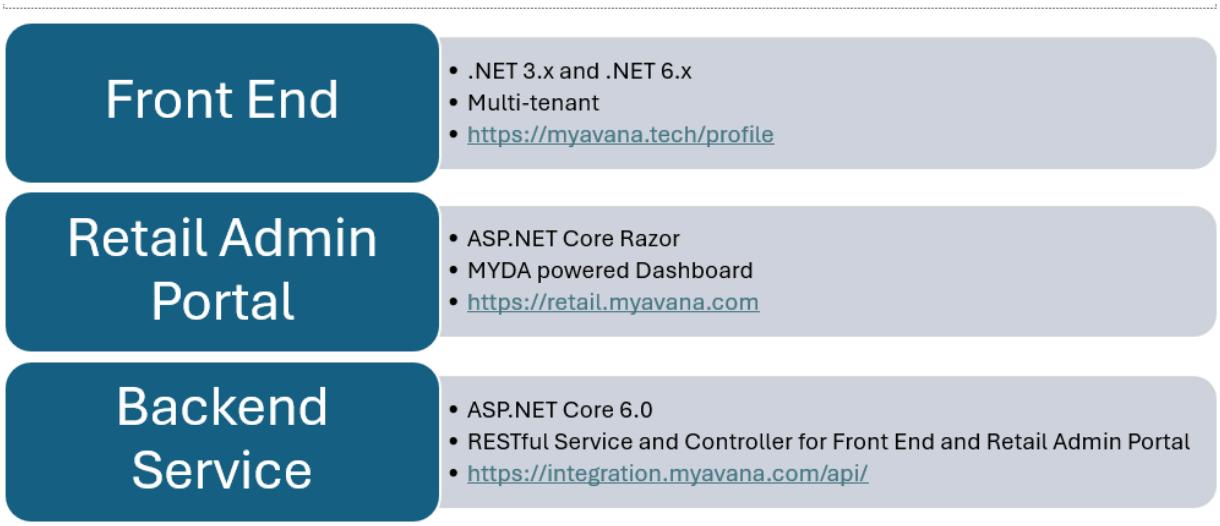

```
[
  {
    "label": "Straight",
    "explainer": "Bulk up straight hair with a protein shampoo and conditioner.",
    "labels": [
      {
        "name": "Straight",
        "value": 99.9601
      },
      {
        "name": "Coily",
        "value": 0.0227535
      },
      {
        "name": "Curly",
        "value": 0.0041467287
```

```
            },
            {
                "name": "Wavy",
                "value": 6.990578E-05
            }
        ],
        "identifier": "Type"
    },
    {
        "label": "Fine",
         "explainer": "For fine textured hair, defrizz with smoothing shampoos and conditioners, deep treatments help with style memory. Bulk up straight fine hair with a protein shampoo and conditioner. Medium Texture - For medium textured hair, balance with moisture and protein conditioners.",
        "labels": [
            {
                "name": "Fine",
                "value": 60.082657
            },
            {
                "name": "Medium",
                "value": 38.511322
            },
            {
                "name": "Coarse",
                "value": 1.4060543
            }
        ],
        "identifier": "Texture"
    }
]
```

MYAVANA Retail Solution

The MYAVANA retail solution is used for integrating with our Business-to-Business (B2B) partners who typically have retail stores (online or brick and mortar). These partners carry products and will like to use our technology (i.e. MYAVANA Hair AI Service) to analyze their customers hair, run a quick survey and recommend products they carry based on the hair type, texture and responses to the survey questions (Goals, Challenges, Product Classification, Style etc.).

This solution is currently designed as a multi-tenancy Software-As-A-Service (SaaS) solution. All partners share the same infrastructure (servers, database) and we delineate them using a TenantId in the database.



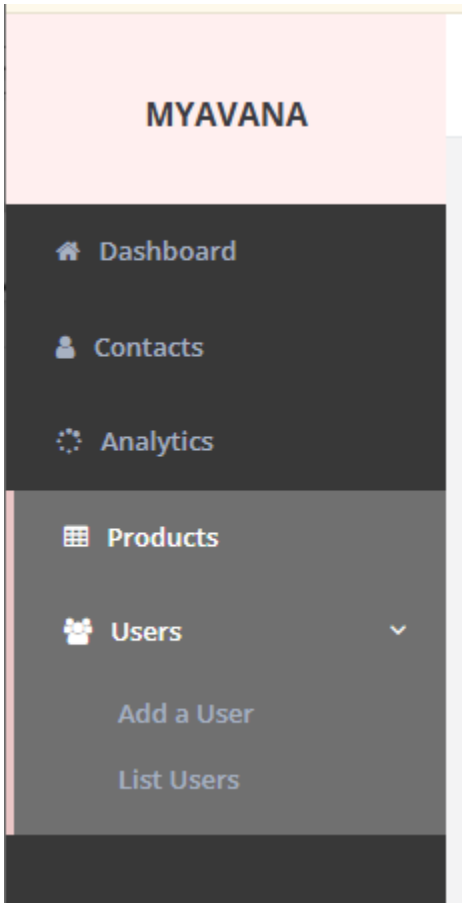
The solution consists of three-components:

1. **Front-end**: This is the multi-tenant mobile/tablet/desktop optimized view which clients share with their end-customers to interact with their tenant. On the front-end, tenants are identified by their “tenant handle”. MYAVANA’s front-end handle is called “profile”. Therefore, to assess the MYAVANA tenant, user will navigate to https://myavana.tech/profile. Tenant handles must be registered and enabled in the backend database for it to render otherwise users will get a message: “This is not a registered Page” when they try to access them. The front-end application is written using react.

2. **Retail Admin Portal**: The backend portal is used by B2B partners to get usage information about their portal. It allows the ability to view their customers that have interacted with their tenant, view results of the Hair AI (i.e. goals, challenges, type, texture and product recommendations), view products they carry and change the displayed name of those products. It also provides a Dashboard as well as analytics based on responses to their MYDA (i.e. response to the survey questions). The portal is hosted at: https://retail.myavana.com. It is developed as an ASP.NET Core Razor project (more in the technical section of this chapter).

   Partners can edit the following attributes of their products:
   - **Name** – the name to be displayed when the product is recommended.
   - **Purchase Link** – The link to the shopping cart where their customers can buy products recommended by HairAI.
   - **Image URL** – A link to the product image to display when recommended by HairAI.

   Menu Options for the portal is illustrated below:



3. **Backend Service**: The backend service acts as the glue logic and contains the business logic that serves both the front-end and retail admin portal. This service is purely restful. It uses a combination of Api Keys (in the request header) and Authorization Token to determine access and whose tenant to grant data access to. The API service is hosted on https://integration.myavana.com/api/ . It is developed on the ASP .NET Core 6.0 framework.

Setting Up New Retail Partners

New Partner setup is a multi-step process which starts by identifying the capabilities desired by the partners. We currently have the following integration options:

HairAI Only Integration:

This is used for partners that already have their internal product recommendation logic (i.e. ULTA) and just needs our analysis API for type/texture detection. In this case, there are two options:

a. **Dedicated HairAI Service Instance:** Typically for enterprise partners with scaling requirements. We will need to provision a new EC2 instance for them, generate an api-key, and create a CNAME for the instance. Once complete, share CNAME, Api-Key and Integration Documentation provided above and also available at: https://docs.google.com/document/d/1PXcDBqTPnJUscMdGdcvenVFXWafR5gPK/edit

Detailed Steps:

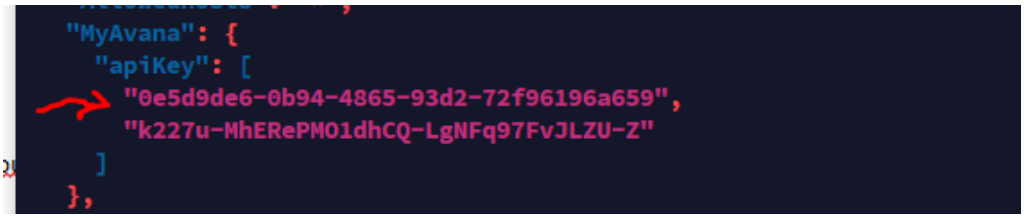
- **Provision New Instance**:
  - Log into AWS Console and select EC2 management (i.e. https://332244720589.signin.aws.amazon.com/console)
  - Select AMIs under Images
  - Right click on “AI AMI Original” and select “Launch Instance from AMI”
  - Give Instance a meaningful name
  - Select Instance type of tx.xlarge (or something larger depending on anticipated workload)
  - Create a New Key pair credential for login
  - Create a new network and obtain a new IP address

    - Set Storage Size to >= 50GB
    - Once complete, select “Launch Instance”
- **Map New CNAME**: once the instance is launched, use the login created (typically RSA as a pem file) and SSH to the server.
    - Open /etc/nginx/sites-available/default
    - Replace server name from “survey.myavana.com” to “{*NEW_TENANT_NAME*}.myavana.com”
    - Remove the SSL mapping from Certbot show below as we will be regenerating this



    - Create a new CNAME by going to https://dash.cloudflare.com/login and logging in
    - Select myavana.com
    - Select DNS
    - Click “Add Record”
    - Select Type as CNAME
    - Put in “{*NEW_TENANT_NAME*}” (as identified above) and for Target, use the IP address of the server i.e. www
    - This step redirects all requests to ““{*NEW_TENANT_NAME*}.myavana.com” to the new server we just created.
- **Add SSL mapping to Server**: this adds security to the server by ensuring all traffic comes via https. To do this, SSH back into the server and type the following command:
    - sudo certbot --nginx -d ““{*NEW_TENANT_NAME*}.myavana.com
        - Note: enter an email address is requested
        - Accept the option to redirect to https
- **Add Api-Key**: Create a random guid to represent this partners API Key

- cd /var/www/apiV2 and edit appsettings.json (i.e. sudo nano appsettings.json)
- replace the myavana -> apiKey section to include this new key.



- **Restart the HairAI service**:
  - sudo systemctl start apiV2.service and sudo systemctl restart nginx.service

b. **Shared HairAI Service:** In this case, the service is already existing. We will be using the service hosted at: https://analysisai01.myavana.com/api/. The steps needed is just to include a new Api-Key and restart the service.

Detailed Steps:

- **SSH to Server**
  - Connect to 52.87.64.1 using the pem file
- **Add New Api Key**
  - Generate a Guid to use as the client api key
  - Update the appSettings.json file located at: /var/www/analysis/appsettings.json to include the api-key on the myavana -> apiKeys node
- **Restart Services**
  - Run the commands: sudo systemctl restart analysis-api.service and sudo systemctl restart nginx.service

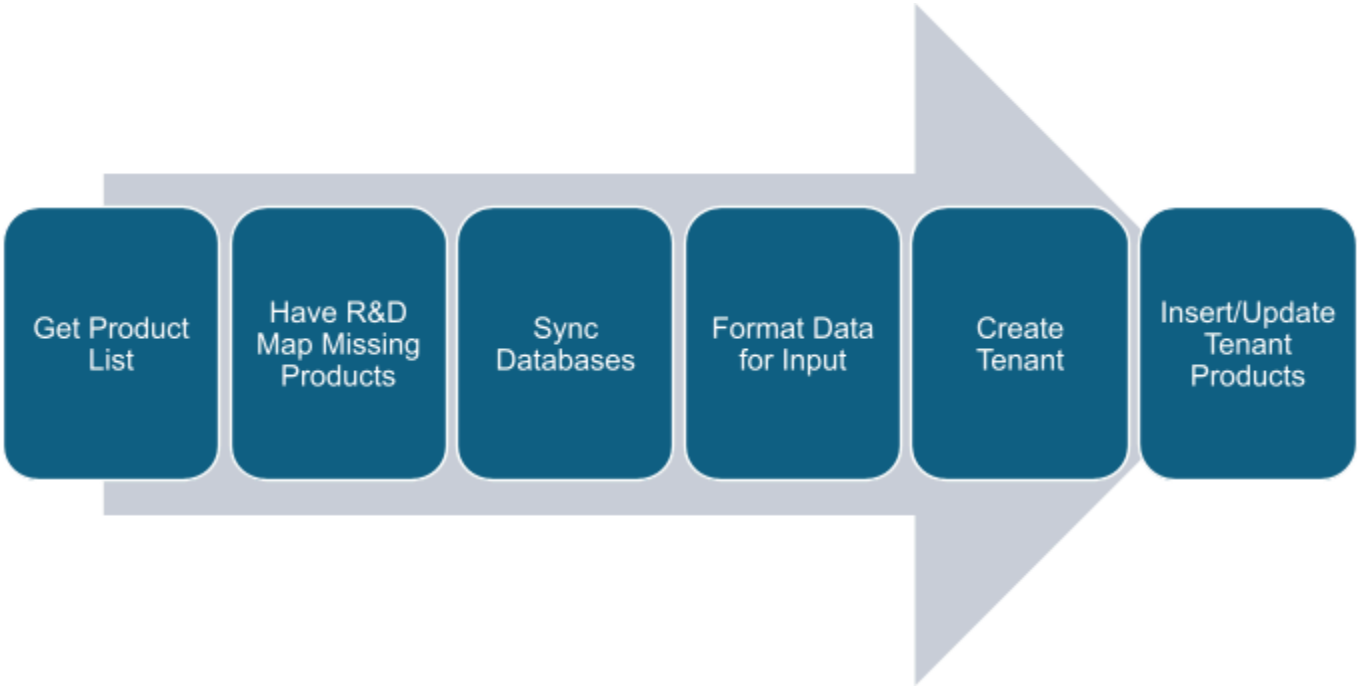
Retail Front-end with Product Integration:

This integration type could be one of three flavors namely:

c. **Integration with Partner Products**: This is for partners who carry their own products. They will have the option to either provide a product link (either a link to buy the product from their website or just a link to the product retailers link which could optionally include an affiliate code).

d. **Integration using MYAVANA Products:** This is for tenants who want to either use all or a subset of the myavana product list. They will typically rely on the default product link we have in our database.

Regardless of where the products come from, onboarding these types of customers follow the same workflow which is illustrated below:



Explanation of Steps:

1. **Get Partner Product List**:

   Ideally, we only need a few fields from customers because all products are already in our database and the data provided by the client is more for their display customization (i.e. what name to display, preferred product image, what URL to go when product is clicked etc.) For this reason, we only require a CSV file with the following columns:

Field Name	Required	Explanation
Product ID	Yes	An internal Id that represents the product on partner system. This could also be the SKU
UPC Code	No	The unique UPC Code of the Product
Name	Yes	Name of the Product

Description	No	Optional description to display about the product
ImageUrl	No	A link to the product image on customer website, if empty we will show our internal one
ProductPageUrl	No	A link to navigate users to when the click on the "Buy Now" link on the retail kiosk. We have the option to either hide this link or redirect users to the products page on the Brands website
SKU	No	An internal SKU of the product

At times, partners provide us a dump of their shopping platform inventory with way more columns than needed and we are left to parse through and identify how to extract the columns above. This could be a value add for when we start to integrate with Point of Sale systems, integrate with 3rd party applications and provide data analysis to Retail partners.

2. **Have R&D Map Missing Products to our Master Database (manual process)**:

   We generally get products list that are difficult to automatically map to our master product database for a couple of reasons such as:
   - New Product we currently don't have
   - Product name is either misspelled or overly abbreviated, so we are unable to deduce what our equivalent product is.

   For these reasons, we are dependent on the R & D team to map those products and provide their corresponding product names. This is typically done by creating a new Tab in the worksheet marked as implementation where the matching product name is entered.
   Note: This is an automation/integration step for us and a new feature for the new SaaS MYAVANA Pro Portal.

3. **Sync Databases (Retail Product Database and Master Product Database) (manual process)**

   Given that the retail product database and our master product database are currently separate we need a mechanism to keep both in sync until we complete implementation of the MYAVANA Pro Portal and database which will unify both platforms. New products entry and product updates are always applied on the master product database. Hence we need to periodically sync this to the retail portal database.

We have a command line program that compares products and brands between both and generates three (3) scripts we need to run against the retail portal database. The scripts do the following:

a. Insert newly added brands and update existing brands
b. Add newly added products (and update existing ones) as well as their mappings to goals, challenges and product classifications
c. Update the products for the MYAVANA tenant of the retail portal (i.e. the /profile tenant)

The database syncing tool is available at https://github.com/MyavanaTechnology/retailPortalDbSync. Pull this project down, update the local paths to save the 3 scripts to and then run it. We can use this tool as part of the new automation/integration feature. Note: This is an automation/integration step for us and a new feature for the new SaaS MYAVANA Pro Portal.

4. **Create Tenant**

Creating a new tenant is currently done directly in the database. Given that the MYAVANA Pro Portal has taken much longer than anticipated, we should add a UI for this in the interim.

We need to add two new Records to both the Tenants and TenantOptions table. The Tenant data contains the basic partner information while the TenantOptions table contains customizations for a tenant. A summary of the columns needed to be filled are as follows:

Tenants Table

- **Id**: Generate a Guid (this will be used as the TenantId and also needs to the added to the appseetings.json file of the HairAI service as detailed in the HairAI section).
- **Company**: Partner's business name
- **TenantName**: this represents the tenant handle to be used to access this tenant on the front-end. i.e. MYAVANA's tenant name is "profile" which allows accessing the tenant via: https://myavana.tech/profile. TenantName MUST contain no space.
- **Address**: Partner business address.
- **ContactFirstName**: First name of Partner contact
- **ContactLastName**: Last name of Partner contact
- **ContactEmail**: Email of Partner
- **IsActive**: Set to true to allow access to this tenant from the front-end

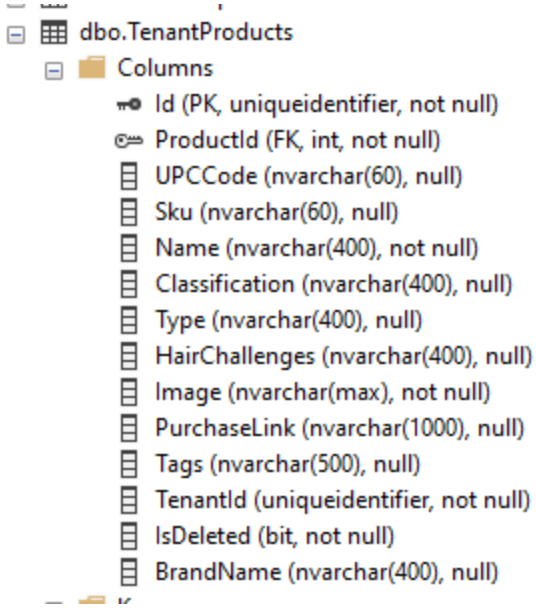
- **IsDeleted**: used to soft delete this partner

TenantOptions Table

- **Id**: Generate a random Guid
- **TenantId**: Enter the Tenant's Id from the Tenant table.
- **LogoLink**: URL to the image to display on the retail portal.
- **Color**: The color for the banner on the retail portal.
- **DateOfBirthRequired**: Determines if Date of Birth is required on questionnaire.
- **ZipCodeRequired**: Determines if zip code is required on questionnaire.
- **ZipCodeLength**: Specifies the zip code length required.
- DataShard: Ignore for now. Will be used for future scaling if we shard the database to support growth.
- **AllowSendResultAsEmail**: Set to true to allow front-end users email their recommended products to an email on the results page.
- **UseProductClassificationFilter**: Set to true to allow user's filter recommended products by classification (i.e. natural or scientifically advanced)
- **UseProductIntersectionOption**: No longer used
- **ShowBuyNowLink**: Set to true to allow user to navigate to the ProductUrl link provided in the TenantProducts table.

5. **Format Data for Input**

Create a CSV to be able to import into the TenantProducts table. This table represents a mapping of the partner's product to our own products which is contained in the **Products** table. It's important to put the same TenantId created in step 4 in the tenantId column of this table. The columns of the CSV should have the following:

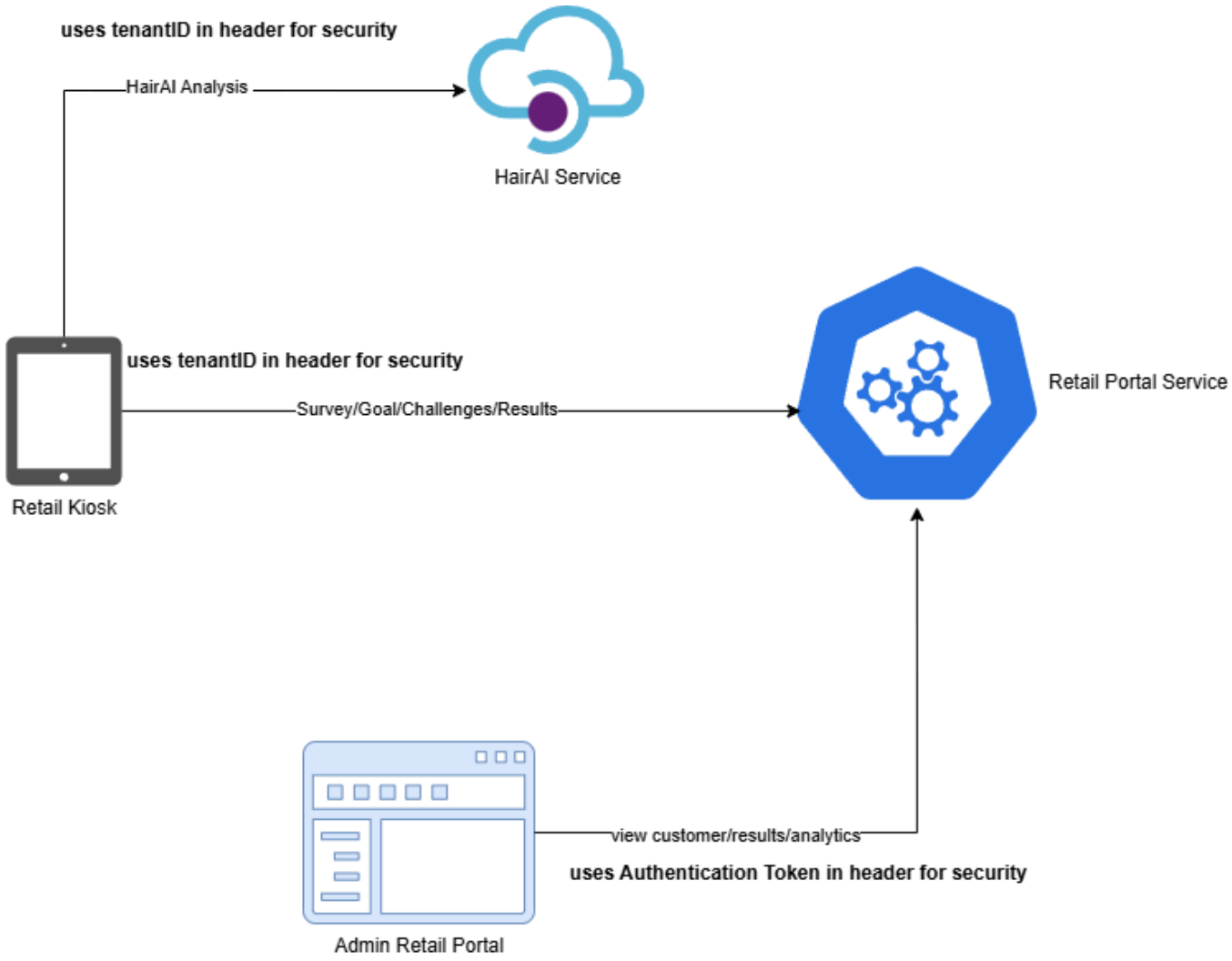


6. **Insert/Update Tenant Products**

   Generate Insert statement to the TenantProduct table from the CSV created in step 5 above. It might be easier to just drop all existing tenant products and then re-insert them. There is no foreign key dependency on this table and thus the flexibility to be able to just drop all tenant products and then re-add them.

High Level Architecture

The interconnectivity of the four components that make up the Retail Portal solution is illustrated below. The front-end (i.e. the Kiosk) allows for taking user image which is sent to the HairAI service. It takes the returned result (type/texture information) along with responses to the survey questions and sends those to the Retail Portal Service which saves that information and recommends products based on this input.



Finally, our partners use the Admin interface of the Retail Portal to view their customers, view their results and also make edits to their product offering.

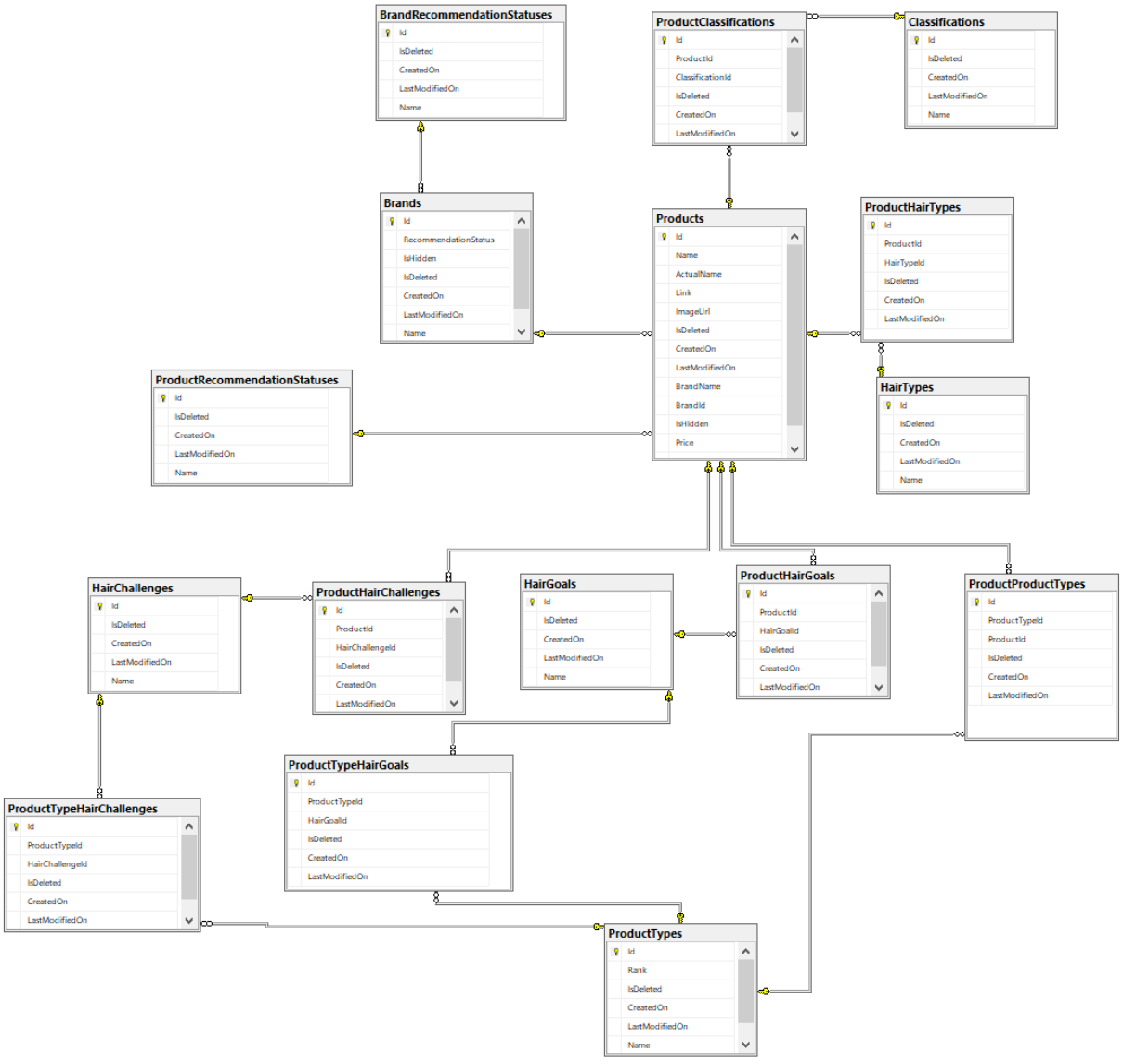
Database Design

The retail portal uses a dedicated database separate from the master database. We have both a production (**name: Avana_B2B_Live**) and staging (**name: Avana_B2B_Staging**) instances.

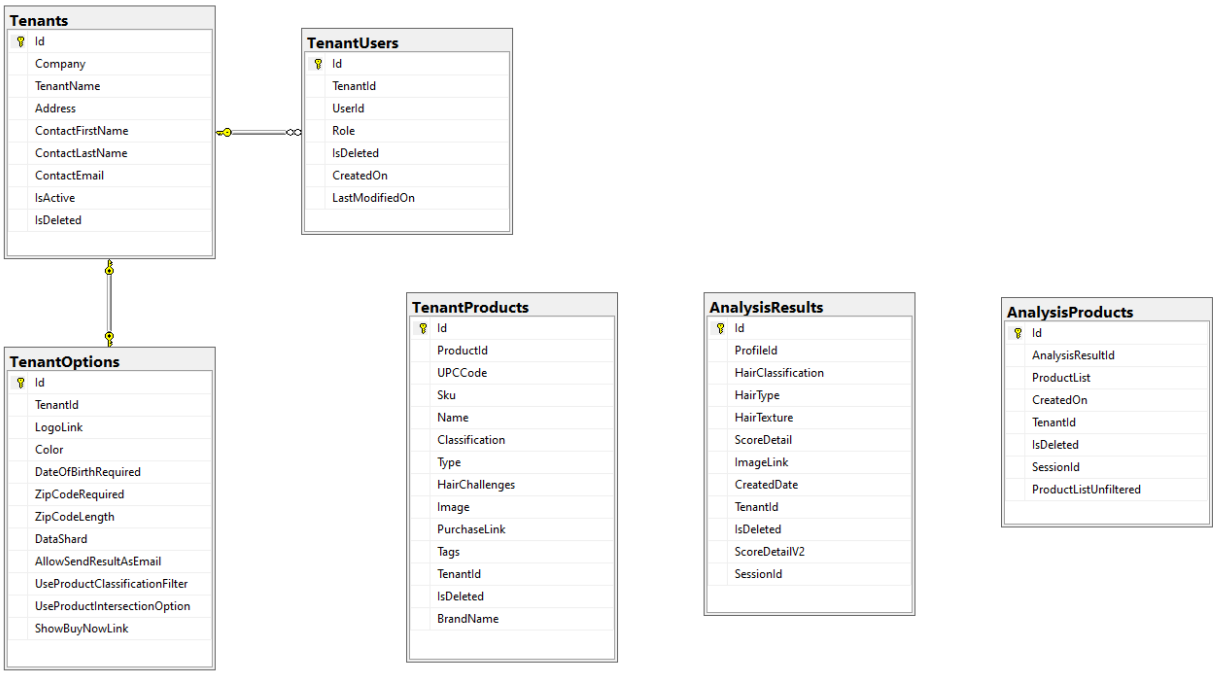
The design for the retail portal can be broadly segmented into three major groups:

1. Product and related attribute definitions (goals, challenges, brands, classifications etc.)
2. Tenant and related attribute definitions (products, customers, results)
3. User management

Product and related attribute definitions



Tenant and related attribute definitions

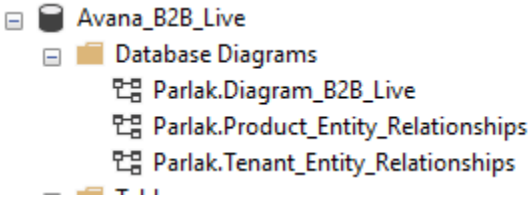


- The **TenantProducts** table holds products a partner carries.
- The **AnalysisResults** table holds the results of the HairAI analysis (i.e. type, texture, link to hair image etc.) for each tenant customer
- The **AnalysisProducts** table holds the products that were recommended for each analysis result (if any)

User management

The user management uses the .NET Core Framework Identity Library for creating users with access to the retail portal.

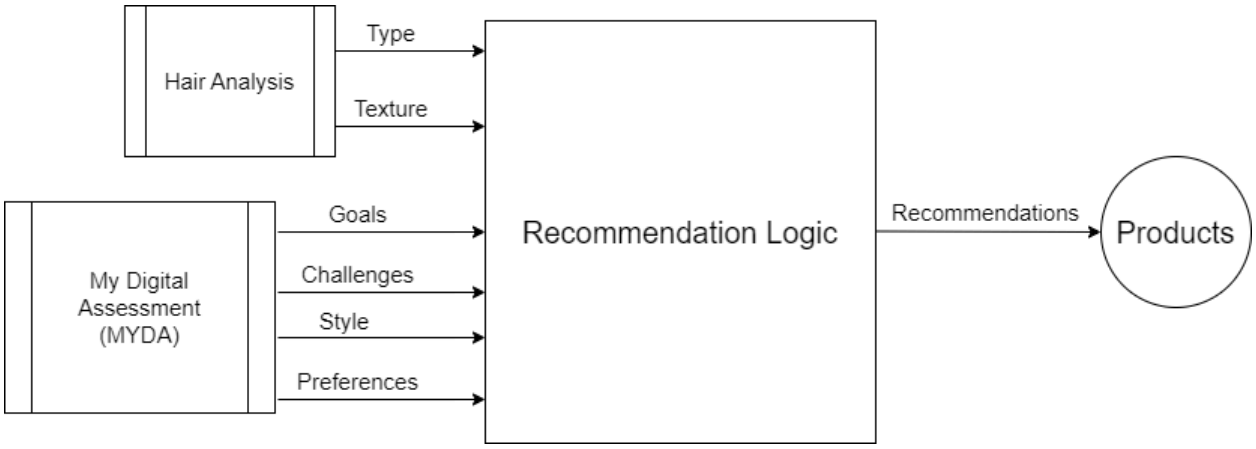
All entity diagrams are maintained on the Production database under the “Database Diagrams” node.



Current Recommendation Logic

This recommendation logic is currently implemented as a Stored Procedure with name: ***sp_GetMatchingProducts_v2***

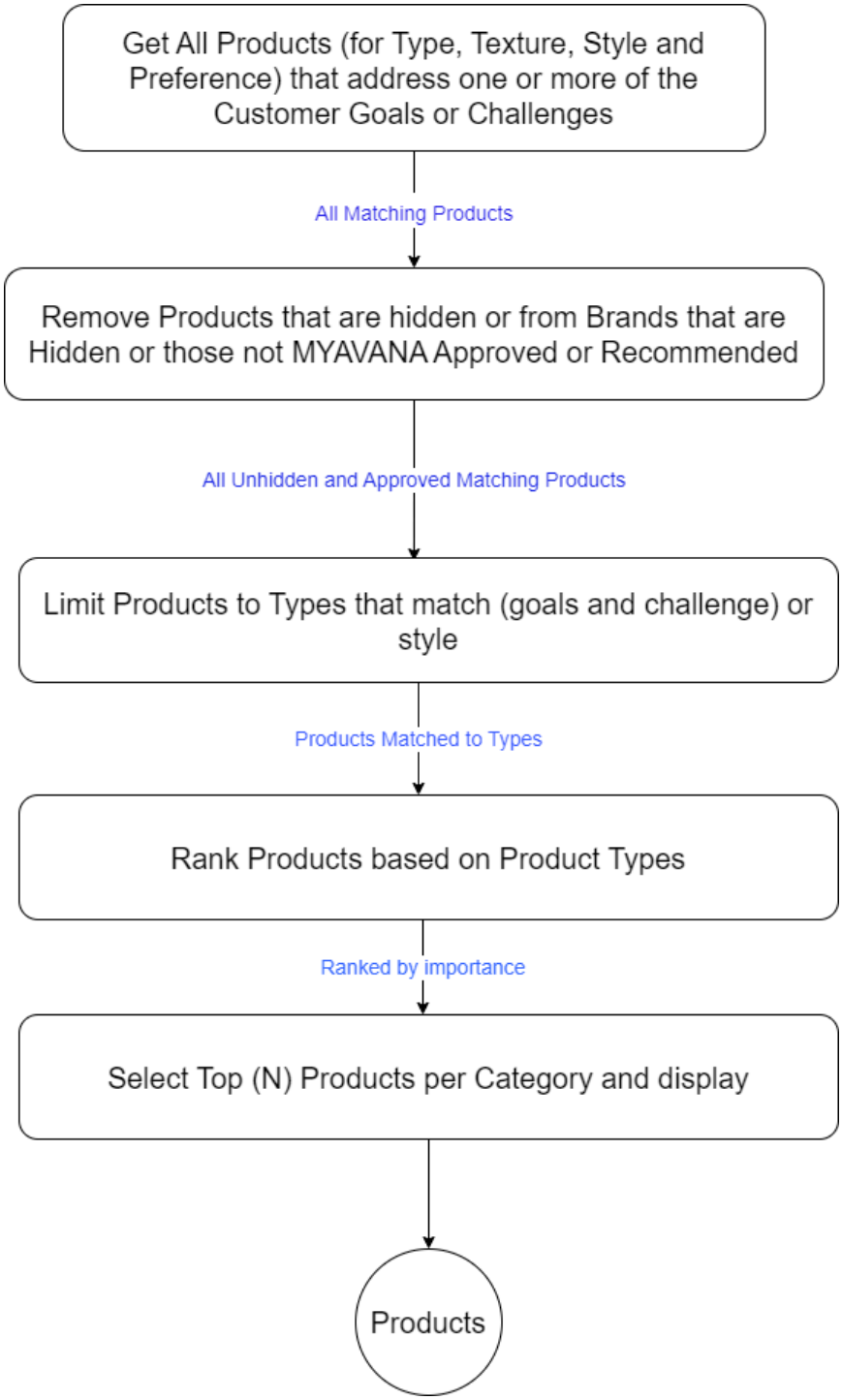
High Level Diagram



Inputs:

- **Type** and **Texture** refers to output of HairAI and is coded. i.e. 3a, 4b etc.
- **Goals** (selected from questionnaire)
- **Challenges** (selected from questionnaire)
- **Current Style** (selected from questionnaire)
- **Preferences** (this is now a post result filtering option). It allows users select products based on
    - **Ingredients**: Natural or Scientifically advanced
    - **Pricing**: This is relative pricing (i.e. $, $$, $$$, $$$$)

Recommendation Logic Flow



Explanation of Algorithm

Step 1:

Products that match Type, Texture AND (Any Goal or Any Challenge or Any Style selected by the user)

Step 2:

Exclude products that are InActive, HiddenInSearch or from Brands that are Hidden or Inactive

- In addition for the MYAVANA Tenant we only include Brand or Products that have the “Myavana Certified” or “MyAvana Approved” status.

Step 3:

Using the spreadsheet provided here https://docs.google.com/spreadsheets/d/1nIni3AOWM2wq1DNFzmERV4gmVm6Lyuwzf68WJ3Km5NQ/edit#gid=1427599582, we only select the Product Types that address the Goal/Challenge.

i.e. if a user Selects Breakage as a Challenge, then only Product types matching (Repairing/Strengthening Shampoo Repairing/Strengthening Treatments Protein Treatment (Strengthening)) will be returned [see Cell B4 in the spreadsheet]. Same Logic will be applied to other goals/challenges.

Products matching user style is also included in the list of products returned.

Step 4:

Again using the Spreadsheet, we will use the columns (Challenge/Goal Challenges) to Rank which product types we return first. So we would prioritize product types in Column B, followed by Column D, then Column F etc. if a product type is in many columns (meaning it addresses more than one challenge or goal, then we will use the highest as the Rank)

Step 5:

Select the Top (3) products for each category as we currently do in the HairAI results. Users can apply preferences as filters on the resulting products page.

Source Code and Design Information

Front-end Source Information

- **Framework**: react
- **Source Location**: https://github.com/MyavanaTechnology/myavana-analysis
- **Latest Branch**: mater
- **Build and Deploy Notes**:
    - **Running Locally**: yarn start (might need to install yarn first and also **run yarn install**)
    - **Deploy Build**: run yarn build.
        - **Ensure that the BASE_API_URL in AppConstants.js file is pointing to correct environment**
            - **Staging:** "https://integrationstaging.myavana.com/api/"
            - **Prod:** "https://integration.myavana.com/api/"
- **Other Notes**: on initial load of front end, a check to made to see if the tenant is valid otherwise it redirects to an error page. As part of this check, the tenant name is encrypted to limit unwarranted DB round-trip to search for the tenant.

Retail Portal Service Source Information

- **Framework**: .NET Core 6
- **Source Location**: https://github.com/MyavanaTechnology/MyAvanaIntegration
- **Latest Branch**: mater
- **Architectural Framework**:
    - The solution is designed around Clean Architecture principles along with CQRS (using the mediatr library). The helps centralize all business logic within the “MyAvana.Survey.Application” project.
    - Database uses EF Core with migrations. Most of the complex queries is implemented as Stored Procedures and uses the “dapper” library as ORM.
        - To generate migrations after domain change
            - open the Package Manager Console (PMC)
            - type: Add-Migration -Name "{GIVE_A_NAME}" -OutputDir "Persistence\Migrations"
        - To update the target database:
            - run command “Update-Database -v” in PMC
- **Build and Deploy Notes**:
    - Update the appsettings.json and appsettings.Production.json files to point to the correct database and corresponding integration endpoint for the environment of interest. Generally, if no changes are made to appSettings file, don’t copy them as part of the deployment.

- Publish the code to the file system and then ftp as detailed in the deployment section below

Admin Portal Source Information

- **Framework**: .NET Core 6
- **Source Location**: https://github.com/MyavanaTechnology/RetailPortalAdmin
- **Latest Branch**: mater
- **Architectural Framework**:
  - The solution is designed around Clean Architecture principles. It connects to the Retail Portal Service exclusively for all backend requests. It doesn't have any database.
- **Build and Deploy Notes**:
  - Update the appropriate appsettings.json, appsettings.Staging.json or appsettings.Production.json file to point to the correct backend service [i.e. either https://integrationstaging.myavana.com/api (for staging) or https://integration.myavana.com/api (for production)]. Generally, if no changes are made to appSettings file, don't copy them as part of the deployment.
  - Before publishing code please comment out the following in Program.cs file. It's a weird bug that needs to be resolved.

```
//#if DEBUG
//app.Run();
//#else
app.Run("http://127.0.0.1:7042");
//#endif
```

  - Publish the code to the file system and then ftp as detailed in the deployment section below
  - This service currently doesn't have a staging environment.

Deployment Information

Staging Environment Deployment

1. **Front-End App**
    - Ensure that the BASE_API_URL in AppConstants.js file is pointing to the correct environment
    - Build the app as defined in the source code section
    - Open an FTP Connection to 52.54.106.156
    - Navigate to /var/www/analysisstaging/
    - Copy the files to this location
    - Restart nginx service (i.e. run command: “sudo systemctl restart nginx.service”)

2. **Retail Portal Service**
    - Update the appsettings.json and appsettings.Production.json files to point to the correct database and corresponding integration endpoint for the target environment
    - Build the app and publish to the file system
    - Open an FTP Connection to 52.54.106.156
    - Navigate to /var/www/integrationstaging/
    - Copy the files to this location
    - Restart the retail portal service (i.e. **run command**: sudo systemctl restart integrationstaging-api.service)
    - Restart nginx service (i.e. **run command**: “sudo systemctl restart nginx.service”)

Production Environment Deployment

1. **Front-End App**
    a. Ensure that the BASE_API_URL in AppConstants.js file is pointing to the correct environment
    b. Build the app as defined in the source code section
    c. Open an FTP Connection to 54.164.3.202
    d. Navigate to /var/www/html/myavanatech
    e. Copy the files to this location
    f. Restart nginx service (i.e. run command: “sudo systemctl restart nginx.service”)

2. **Retail Portal Service**

a. Update the appsettings.json and appsettings.Production.json files to point to the correct database and corresponding integration endpoint for the target environment
b. Build the app and publish to the file system
c. Open an FTP Connection to 54.164.3.202
d. Navigate to /var/www/html/b2bapiservice
e. Copy the files to this location
f. Restart the retail portal service (i.e. **run command**: sudo systemctl restart b2bapiservice.service)
g. Restart nginx service (i.e. **run command**: “sudo systemctl restart nginx.service”)

3. **Retail Admin Portal App**
    a. Update the appropriate appsettings.json, appsettings.Staging.json or appsettings.Production.json file to point to the correct backend service for the target environment
    b. Comment out the app.Run() component described in the source code section above
    c. Build the app and publish to the file system
    d. Open an FTP Connection to 54.164.3.202
    e. Navigate to /var/www/html/retailportal
    f. Copy the files to this location
    g. Restart the retail admin portal app (i.e. **run command**: sudo systemctl restart retailportal.service)
    h. Restart nginx service (i.e. **run command**: “sudo systemctl restart nginx.service”)

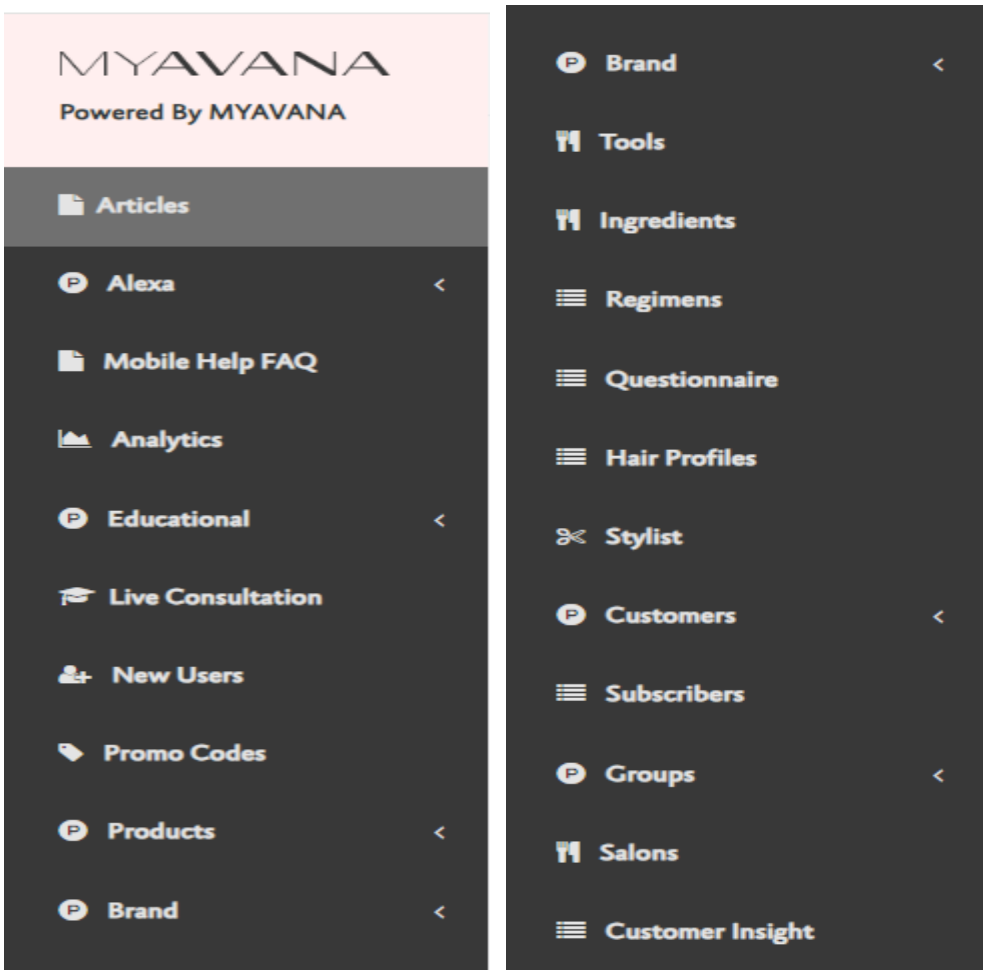
Other Notes:

- Site definitions are available at: /etc/nginx/sites-available

MYAVANA Solution

Admin Portal Overview

The **Admin Panel** is a centralized management tool for administrators to oversee and control various aspects of the application. It provides essential features like user management,analytics , products , tools , groups , salons, and managing customer hair profile. It provides a comprehensive interface for administrators to organize and monitor all aspects of the platform, from content management and user profiles to promotional tools and analytics.Admin Portal also provides a crucial Role Management feature. This functionality allows administrators to assign and modify roles for different users within the platform. Roles help control access to various features and ensure that users have appropriate permissions based on their responsibilities.It typically features different sections that correspond to various platform functions, each offering specific controls for adding, editing, deleting, or viewing content and data.



The **Admin Portal** provides a comprehensive suite of tools and features to help administrators effectively manage the platform and its various components. Key functionalities include:

- **Analytics**: This section allows admins to view detailed platform analytics and insights, helping to track user behavior, content performance, and overall platform activity. This data is crucial for making informed decisions and optimizing the platform.
- **Educational**: The Educational Videos or tips section allows administrators to manage and organize video content effectively. Admins can add descriptions, assign titles, and categorize videos for easy discovery. They can also toggle the **"**Show On Mobile**"** option to control visibility on mobile devices, ensuring an optimized experience for all users.
- **New Users**: This functionality allows administrators to assign and modify roles for different users within the platform.
- **Promo Codes**: This section allows the creation and management of promotional codes and discounts, helping admins run marketing campaigns or special offers for users.
- **Products**: Admins can manage product listings, update associated details, and ensure that products are categorized and displayed correctly on the platform.
- **Brand**: This feature enables the management of brand information, including details about various product brands featured on the platform.
- **Tools**: Admins can add, edit, and delete tools as needed. This section allows admins to add detailed information about various tools related to different types of hair health. Each tool can be categorized and described based on its intended use, ensuring that users have access to the right tools for their specific hair care needs.
- **Ingredients**: Admins can manage ingredient details, including the Name, Type, Image, Description, and the specific Hair Challenges the ingredient addresses. For example, Rice Protein is a natural ingredient that improves hair volume, adds shine, and helps with issues like dry hair, dry scalp, and damage from heat. This section provides important information about how ingredients benefit various hair types and challenges.
- **Regimens**: Admins can manage and create personalized hair care routines for various hair challenges. For example, for Brittle, Wiry Hair, a regimen titled Brittle Hair Solutions can be provided. This regimen addresses common causes of brittle hair, such as sun exposure, heat styling, humidity, and smoking, offering solutions to restore moisture and improve hair strength. Admins can create, edit, and delete regimens to cater to different hair care needs.

- **Questionnaire**: In the Admin Portal, under the Questionnaire section, admins can view the questionnaire filled out by customers. This data is used to manually create the Healthy Hair Care Plan (HHCP) or to automatically generate it, providing a personalized hair care plan for the customer based on their responses.
- **Hair Profiles**: This feature allows admins to see all the customers the Healthy Hair Care Plan (HHCP).
- **Stylist**: Admins have the ability to manage stylists. This includes the ability to add, edit, and delete stylist information.
- **Customers**: Admins have the ability to add, edit, and delete customer profiles. Additionally, admins can directly fill out the customer's questionnaire if required. This feature allows admins to complete the questionnaire on behalf of the customer, ensuring the necessary data is collected and used to create a personalized Healthy Hair Care Plan (HHCP).
- **Subscribers**: This section enables the management of user subscriptions, including newsletter sign-ups and other communication channels to engage with users.
- **Groups**: In the Groups section of the Admin Portal, admins can add, edit, and delete groups. Groups can be classified as private or public. If a group is marked as private, users must send a request to join, which then requires approval. For public groups, users can join directly without needing approval.
- **Salons**:Admins have the ability to manage Salons. This includes the ability to add, edit, and delete salons information like name or address.
- **Customer Insight**: Admins can access customer feedback and insights to track user satisfaction, identify trends, and make data-driven improvements to the platform and services.

Backend

The backend service is the core logic layer connecting the front-end application and admin portal, developed with ASP.NET Core 2.2.

1. **MyAvana.AI.Api**: This microservice is likely responsible for the artificial intelligence (AI) functionalities of the platform. It could handle tasks such as digital hair analysis.

2. **MyAvana.Auth.Api**: This microservice manages authentication and authorization. It handles user login, registration, session management, and ensuring that only authorized users can access specific resources or perform certain actions. It may

also manage API keys, tokens, and roles for secure access.

3. **MyAvana.CRM.Api**: This microservice manages customers related data, salons, and admin operations. It manages customer profiles, including hair data and health plans, tracks salon details like services and staff, and supports admin functionalities such as managing user roles, permissions, and interactions. It centralizes customer, salon, and admin data to streamline interactions, optimize user experiences, and provide insights for business decisions.

4. **MyAvana.Payments.Api**: This microservice handles all payment-related processes.

High Level Architecture

Please refer to the approach on the retail portal section above to describe the High Level Architecture and how all the microservices connect. Include details of the APi gateway and draw a diagram.

1. Frontend and API Gateway

- **Frontend**: Web portal and mobile apps interact with the backend via the API Gateway.
- **MyAvana.Gateway**:
    - Manages incoming requests and routes them to appropriate services using configurations defined in **Ocelot.json**.
    - Implements authentication and authorization for secured access.

2. Microservices

Each microservice is independent and provides a specific functionality:

1. **MyAvana.AI.Api**: Manages AI-related features (e.g., recommendations or analytics).
2. **MyAvana.Auth.Api**: Handles user authentication and token management.
3. **MyAvana.CRM.Api**: Manages customer relationship and profile data.
4. **MyAvana.Payments.Api**: Manages payment processing and integrations.

3. Utilities

- **MyAvana.Crypto**: Likely provides encryption/decryption functionalities for sensitive data.
- **MyAvana.EmailService**: Handles email notifications (e.g., for registration, orders, or promotions).
- **MyAvana.Framework**: Contains reusable frameworks or utility methods shared across services.
- **MyAvana.Logger**: Manages logging, possibly integrated with centralized tools (e.g., ELK stack).

4. Data Layer

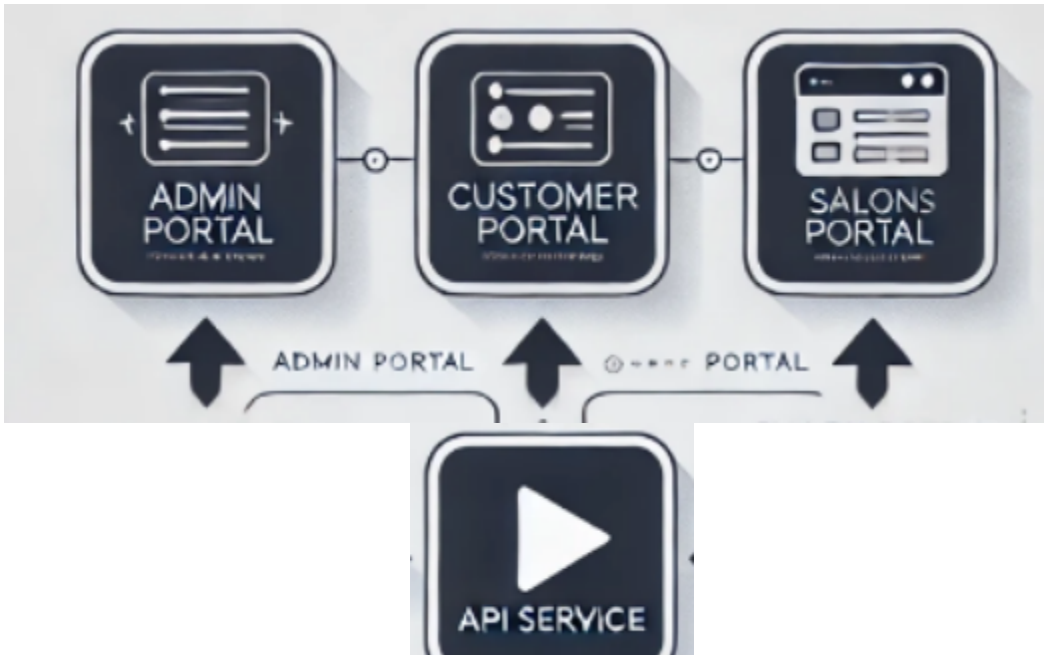
- **MyAvana.DAL**: Centralized Data Access Layer, abstracting database interaction for shared entities or cross-service data operations.
- **EFCoreDemoContext.cs**: Represents the Entity Framework Core data context for database interactions.

5. Additional Component: MyAvana.ExcelImport

- Likely used for importing data from Excel files into the system.
- Includes ExcelFiles for file processing and EFCoreDemoContext.cs for database context.

Connection Between Components

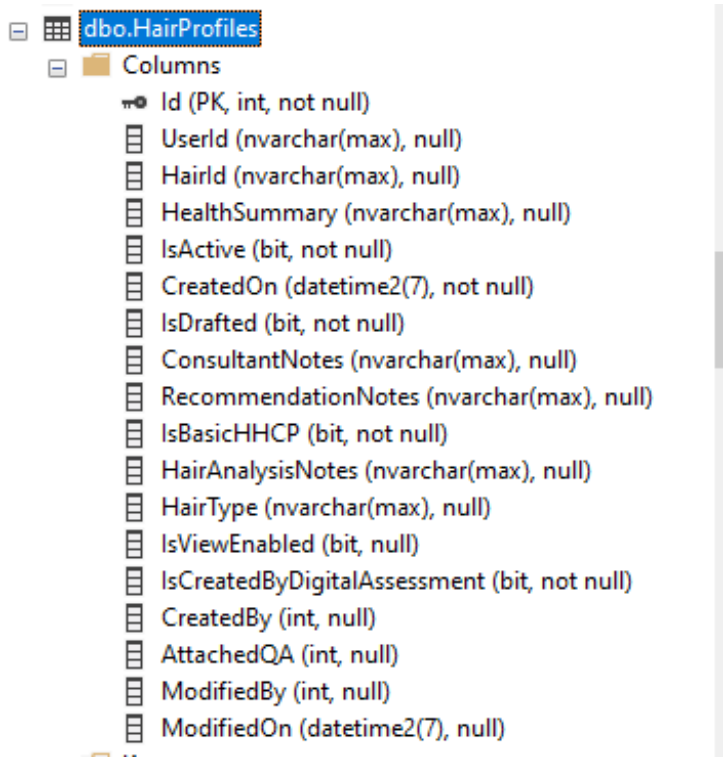
- **Clients (Web/Mobile)** → **API Gateway** (Ocelot) → **Microservices**.
- Each microservice operates independently with its own database or uses the DAL for shared data access.
- Shared utilities like EmailService, Logger, and Crypto are used by the microservices.
- ExcelImport can interact with microservices or the DAL to insert or process imported data.



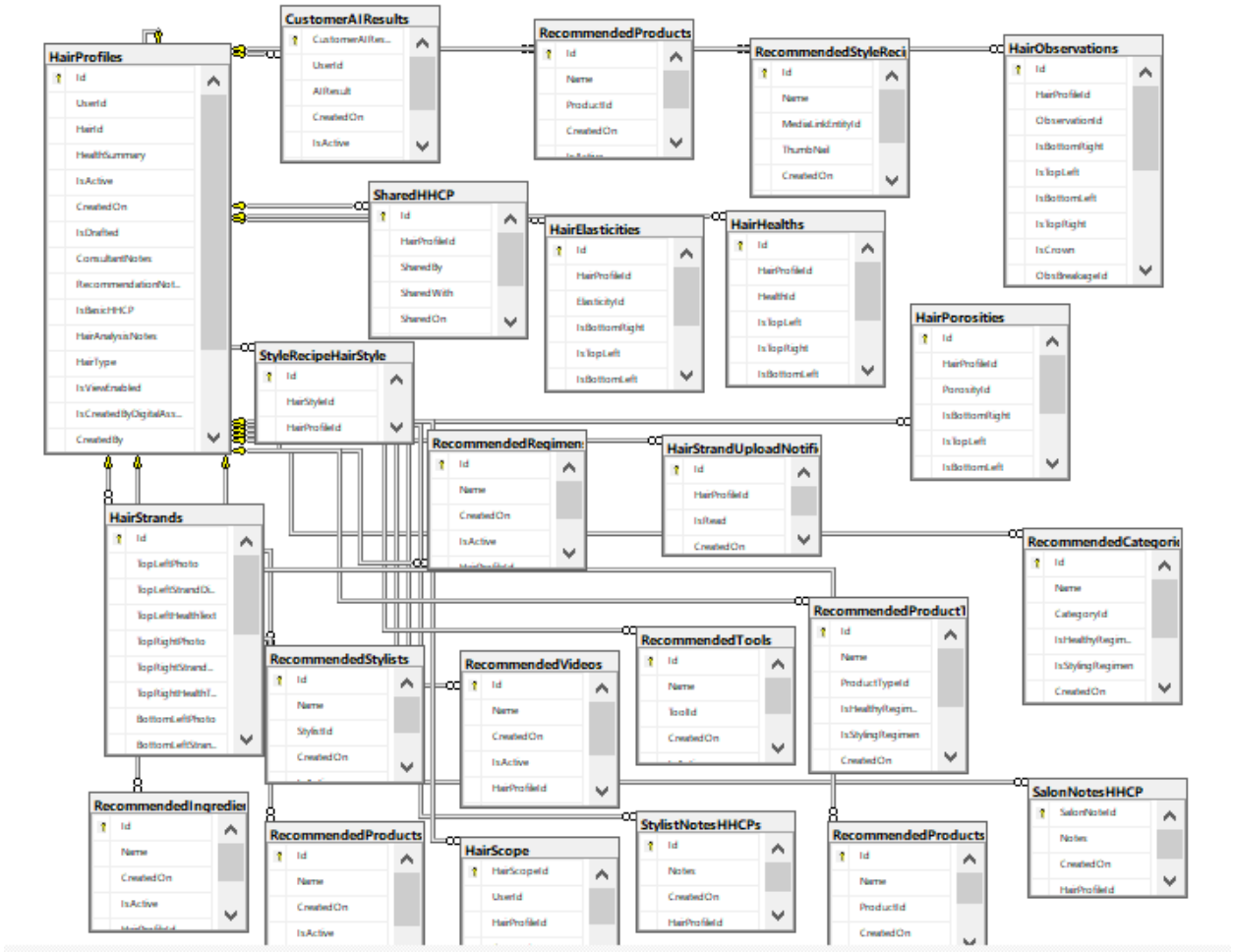
Database Design

Please refer to the approach on the retail portal section above to break down the data model along major Entities (Product/HHCP, MYDA etc). Generate database diagrams to show tables used by major entities.

Hair Profile

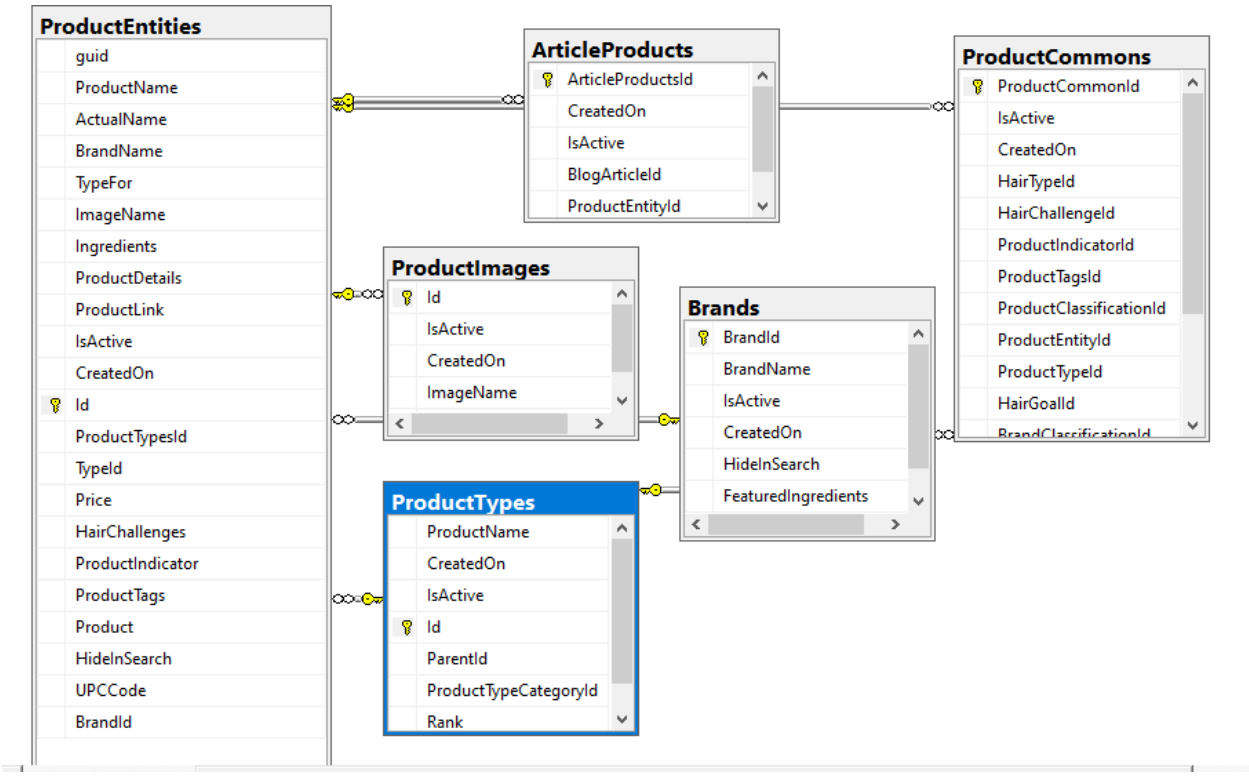


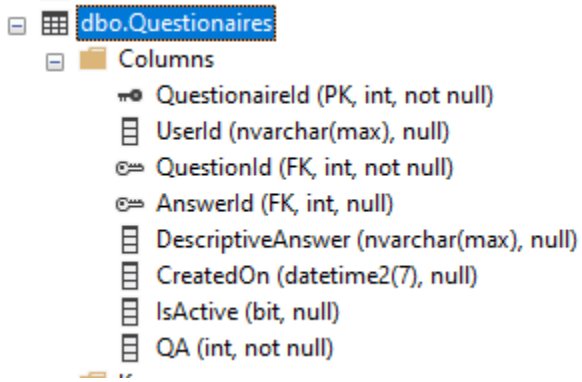
Database diagram


HairProfiles
Id
UserId
HairId
HealthSummary
IsActive
CreatedOn
IsDrafted
ConsultantNotes
RecommendationNot...
IsBasicHHCP
HairAnalysisNotes
HairType
IsViewEnabled
IsCreatedByDigitalAss...
CreatedBy
CustomerAIResults
CustomerAIRes...
UserId
AIResult
CreatedOn
IsActive
RecommendedProducts
Id
Name
ProductId
CreatedOn
RecommendedStyleReci
Id
Name
MediaLinkEntityId
ThumbNail
CreatedOn
HairObservations
Id
HairProfileId
ObservationId
IsBottomRight
IsTopLeft
IsBottomLeft
IsTopRight
IsCrown
ObsBreakageId
SharedHHCP
Id
HairProfileId
SharedBy
SharedWith
SharedOn
HairElasticities
Id
HairProfileId
ElasticityId
IsBottomRight
IsTopLeft
IsBottomLeft
HairHealths
Id
HairProfileId
HealthId
IsTopLeft
IsTopRight
IsBottomLeft
HairPorosities
Id
HairProfileId
PorosityId
IsBottomRight
IsTopLeft
IsBottomLeft
StyleRecipeHairStyle
Id
HairStyleId
HairProfileId
RecommendedRegimen
Id
Name
CreatedOn
IsActive
HairStrandUploadNotifi
Id
HairProfileId
IsRead
CreatedOn
HairStrands
Id
TopLeftPhoto
TopLeftStrandOr...
TopLeftHealthText
TopRightPhoto
TopRightStrand...
TopRightHealthT...
BottomLeftPhoto
BottomLeftStran...
RecommendedCategorie
Id
Name
CategoryId
IsHealthyRegim...
IsStylingRegimen
CreatedOn
RecommendedProductT
Id
Name
ProductTypeId
IsHealthyRegim...
IsStylingRegimen
CreatedOn
RecommendedStylists
Id
Name
StylistId
CreatedOn
RecommendedVideos
Id
Name
CreatedOn
IsActive
HairProfileId
RecommendedTools
Id
Name
ToolId
CreatedOn
RecommendedIngredie
Id
Name
CreatedOn
IsActive
RecommendedProducts
Id
Name
CreatedOn
IsActive
HairScope
HairScopeId
UserId
HairProfileId
StylistNotesHHCPs
Id
Notes
CreatedOn
HairProfileId
RecommendedProducts
Id
Name
ProductId
CreatedOn
SalonNotesHHCP
SalonNoteId
Notes
CreatedOn
HairProfileId

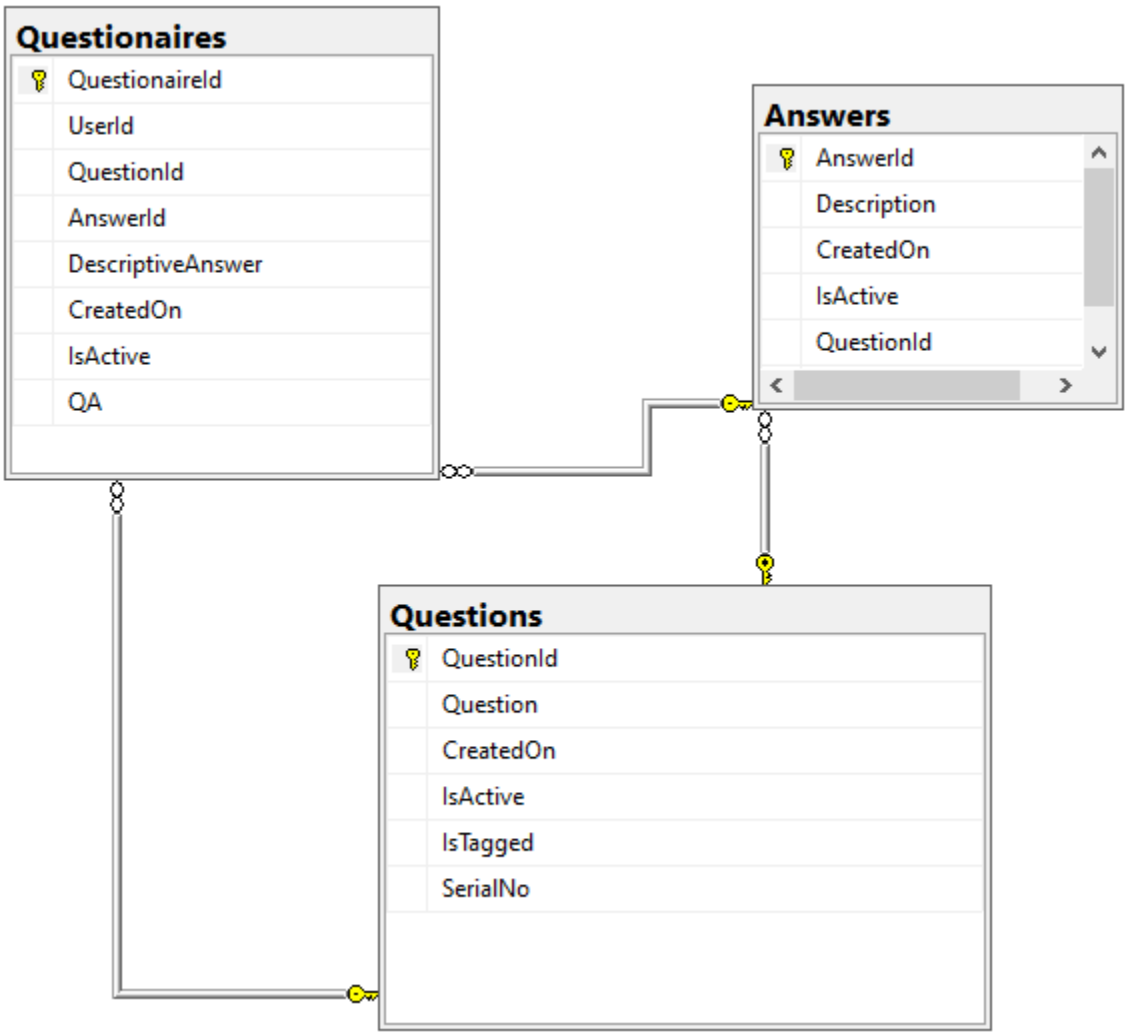
- dbo.ProductEntities
  - Columns
    - guid (uniqueidentifier, not null)
    - ProductName (nvarchar(max), null)
    - ActualName (nvarchar(max), null)
    - BrandName (nvarchar(max), null)
    - TypeFor (nvarchar(max), null)
    - ImageName (nvarchar(max), null)
    - Ingredients (nvarchar(max), null)
    - ProductDetails (nvarchar(max), null)
    - ProductLink (nvarchar(max), null)
    - IsActive (bit, not null)
    - CreatedOn (datetime2(7), null)
    - Id (PK, int, not null)
    - ProductTypesId (FK, int, null)
    - TypeId (int, null)
    - Price (decimal(18,2), not null)
    - HairChallenges (nvarchar(max), null)
    - ProductIndicator (nvarchar(max), null)
    - ProductTags (nvarchar(max), null)
    - Product (nvarchar(max), null)
    - HideInSearch (bit, null)
    - UPCCode (nvarchar(max), null)
    - BrandId (FK, int, null)

Database diagram





Database diagram



Product Recommendation Logic and Auto HHCP Logic

Please refer to the approach on the retail portal section above, discuss the current recommendation logic for products and videos. Provide a break down and point to the where this is implemented (either code file name and function or Stored Procedure)

Auto Generate HHCP

1. sp_GetMatchingProducts

Purpose: The **sp_GetMatchingProducts** stored procedure is designed to recommend products based on a user's hair profile, hair challenges, goals, product preferences, and hair styles. The procedure matches products from the product catalog that meet the criteria defined by the user's answers in the latest questionnaire.

Input Parameters:

- @HairType: A string representing the user's hair type (e.g., curly, straight, etc.) used for filtering the matching products.
- @UserID: A unique identifier for the user whose product recommendations are being generated.
- @QA: The optional questionnaire identifier. If not provided, the latest questionnaire answer for the user is used.

Logic Breakdown:

1. Retrieving the Latest Questionnaire Data:
    - If @QA is provided, it is used to fetch the latest answers from the user.
    - If @QA is not provided, the procedure retrieves the most recent questionnaire for the user by querying the Questionaires table.
2. Temporary Tables Creation: Temporary tables are created to store user-specific data for hair challenges, goals, product preferences, and hair styles.
    - #Challenges: Stores the user's selected hair challenge.
    - #Goals: Stores the user's selected hair goals.
    - #Classifications: Stores the user's selected product preferences.
    - #HairStyles: Stores the user's selected hair styles.
    - #HairTypesId: Stores the hair types based on the input @HairType.
3. Mapping User Selections to Products:
    - The procedure queries various product-related tables (ProductCommons, ProductEntities) to filter products based on the user's hair type, challenges, goals, classifications, and hair styles.
    - The relevant product information (name, brand, hidden status, etc.) is inserted into a temporary table #UnfilteredMatchingProducts.

4. Ranking and Filtering Products:
    - The procedure ranks products based on relevance using a ROW_NUMBER function, considering product type, challenge, and goal mappings.
    - The final result is limited to the top 3 matching products based on the rankings.
5. Final Output:
    - The procedure returns the top 3 products along with their name, actual name, ranking, product type, and brand information.
    - Products are ranked by their relevance, considering matching challenges, goals, and styles.
6. Temporary Tables Cleanup:
    - After executing the logic, all temporary tables (#Challenges, #Goals, #Classifications, #HairStyles, #HairTypesId, #ProductProductTypes, #UnfilteredMatchingProducts) are dropped to ensure no residual data remains in the system.

AutoRecommend Videos

Purpose: The AutoRecommendVideos stored procedure is designed to automatically recommend relevant videos for a user based on their hair profile, challenges, goals, and scalp concerns. It pulls data from the user's latest questionnaire responses and matches it with available videos linked to the user's hair challenges and goals.

Input Parameters:

- @HairProfileId: A unique identifier representing the user's hair profile, which helps personalize the video recommendations.
- @UserID: A unique identifier for the user whose videos are being recommended.
- @QA: An optional questionnaire identifier. If not provided, the procedure uses the latest active questionnaire data for the user.

Logic Breakdown:

1. Retrieve the Latest Questionnaire Data:
    - If @QA is provided, it is used to retrieve the user's latest questionnaire.
    - If not, the procedure fetches the most recent active questionnaire by querying the Questionaires table.
2. Temporary Tables Creation: Temporary tables are created to store data about the user's selected hair challenges, goals, and scalp challenges:
    - #Challenges: Stores the user's selected hair challenges.
    - #Goals: Stores the user's selected hair goals.
    - #ScalpChallenges: Stores the user's selected scalp challenges.

3. Mapping User Selections to Videos:
    - The procedure queries the MediaLinkEntities table to find videos linked to the user's selected hair challenges, goals, and scalp challenges using the HairGoalVideoMapping and HairChallengeVideoMapping tables.
    - Videos that match the selected challenges and goals are retrieved.
4. Inserting Recommended Videos:
    - Relevant videos are inserted into the RecommendedVideos table. This includes video details (name, media entity ID, and the user's hair profile ID).
    - The video recommendations are based on the intersection of the selected challenges and goals.
5. Temporary Tables Cleanup:
    - Once the recommendations are generated and inserted into the RecommendedVideos table, all temporary tables (#Challenges, #Goals, #ScalpChallenges) are dropped to maintain system performance and avoid lingering data.

3. sp_GetStyleRecipeProducts

Purpose: The **sp_GetStyleRecipeProducts** stored procedure is designed to recommend personalized hair care products to a user based on their hair profile, challenges, goals, preferences, and selected hairstyles. It uses the latest questionnaire responses to filter and rank products that best match the user's needs and preferences.

Input Parameters:

- @HairType: A comma-separated string representing the user's hair type (e.g., curly, straight, wavy). This parameter is used to filter products based on the user's hair type.
- @UserID: A unique identifier for the user whose products are being recommended.

Logic Breakdown:

1. Retrieve Latest Questionnaire Data:

- The procedure starts by determining the most recent active questionnaire for the user by querying the Questionaires table. It fetches the latest questionnaire version (@MaxQA) to ensure recommendations are based on the most up-to-date user responses.

2. Temporary Tables Creation:

- Temporary tables are created to store data about the user's selected preferences:
    - #Challenges: Stores the hair challenges selected by the user.
    - #Goals: Stores the hair goals selected by the user.
    - #Classifications: Stores the product classifications based on the user's preferences.
    - #HairStyles: Stores the hairstyles selected by the user.
    - #HairTypesId: Stores the corresponding hair type IDs based on the input @HairType.

3. Mapping User Selections to Temporary Tables:

- The procedure inserts the user's selections into the corresponding temporary tables:
    - Hair Challenges: Retrieved using HairChallengeId based on the user's answers in the questionnaire.
    - Hair Goals: Retrieved using HairGoalId based on the user's answers.
    - Product Preferences (Classifications): Retrieved using ProductPreferenceId based on the user's answers.
    - Hair Styles: Retrieved using HairStyleId based on the user's answers.
    - Hair Types: Based on the provided @HairType input, the procedure retrieves the corresponding hair type IDs from the HairTypes table.

4. Create Product-Product Type Relationship:

- A temporary table #ProductProductTypes is created to establish the relationship between products and product types (using the ProductCommonId, ProductEntityId, and ProductTypeId).

5. Identify Matching Products:

- The procedure identifies products that match the user's selected challenges, goals, hair types, preferences, and styles. The matching products are inserted into the #UnfilteredMatchingProducts table. It tracks the number of matches for each product based on the user's profile and selection.

6. Rank and Filter Products:

- The products are ranked using the ROW_NUMBER() function, which orders products by the best match for the user's profile (prioritizing rank and relevance). The query ensures that products that are active, visible to the user, and aligned with the user's hair care goals are included.

7. Return Top 3 Products:

- After ranking, the procedure selects the top 3 products for each product type, based on the highest number of matches and the rank. These top products are returned to the user.

8. Output:

- The procedure returns the following details for each recommended product:
    - Id: The product identifier (ProductEntityId).
    - ProductName: The name of the product.
    - ActualName: The actual name of the product.
    - Matches: The number of matching criteria for the product.
    - Rank: The rank of the product based on relevance.
    - ProductType: The type of the product.

9. Temporary Tables Cleanup:

- After generating the recommendations, all temporary tables (#Challenges, #Goals, #Classifications, #HairStyles, #HairTypesId, #ProductProductTypes, and #UnfilteredMatchingProducts) are dropped to ensure no residual data affects system performance.

Source Code and Design Information

Similar to the retail portal section above, discuss each microservice, framework, source code location, current branch, architectural peculiarities, and any notes of interest. Include how to build and deploy and what environment variables are needed

Admin Portal Source Information

- **Framework**: .NET Core 2.2
- **Source Location**: https://github.com/MyavanaTechnology/MyavanaAdminV2
- **Latest Branch**: mater
- **Architectural Framework**:
    - The solution is designed around Clean Architecture principles. It connects to the Myavana Service exclusively for all backend requests. It doesn't have any database.
- **Build and Deploy Notes**:
    - Update the appropriate appsettings.json to point to the correct backend service [i.e. either https://apistaging.myavana.com/ (for staging) or https://api.myavana.com/ (for production)]. Generally, if no changes are made to appSettings file, don't copy them as part of the deployment.
    - Publish the code to the file system

- This portal has a staging environment as well as production.

Myavana Backend Service Source Information

- **Framework**: .NET Core 6
- **Source Location**: https://github.com/MyavanaTechnology/MyAvana_Backend_ASP
- **Latest Branch**: master
- **Architectural Framework**:
  - The solution is designed around Clean Architecture principles.
  - Database uses EF Core with migrations. Most of the complex queries is implemented as Stored Procedures and uses the “dapper” library as ORM.
    - To generate migrations use Auth API
      - open the Package Manager Console (PMC)
      - type: Add-Migration -Name "{GIVE_A_NAME}"
      - To update the target database: run command “Update-Database
- **Build and Deploy Notes**:
  - Update the appsettings.json to point to the correct database(staging or production). Generally, if no changes are made to appSettings file, don’t copy them as part of the deployment.
  - Publish the code to the file system(different folder for each microservice)
  - This service has a staging environment as well as production.

Deployment Information

Staging Environment Deployment

1. **Admin Portal**
   - Ensure that the WebApiBaseUrl in AppSettings.json file is pointing to the correct environment to staging api url(https://apistaging.myavana.com/)
   - Build the app as defined in the source code section
   - Open an FTP Connection to 52.54.106.156
   - Navigate to /var/www/adminstaging/
   - Copy the files to this location
   - Restart the admin portal service (i.e. run command: “sudo systemctl restart adminstaging.service”)

2. **Backend service**

- Update the appsettings.json point to staging database(Avana_Live_Staging).
- Build each microservice app and publish to the file system(separate folder)
- Open an FTP Connection to 52.54.106.156
- Navigate to /var/www/auth/, var/www/crm,var/www/gateway/,var/www/payments/
- Copy the files to this location for appropriate microservice
- Restart the require service
- (i.e. **run command**:
  Crm: sudo systemctl restart avanacrm.service
  Auth: sudo systemctl restart avanaauth.service
  Payments: sudo systemctl restart avanapayment.service
  Gateway: sudo systemctl restart avanagateway.service
- Restart nginx service (i.e. **run command**: “sudo systemctl restart nginx.service”)

Production Environment Deployment

3. **Admin Portal**
    - Ensure that the WebApiBaseUrl in AppSettings.json file is pointing to the correct environment to staging api url(https://api.myavana.com/)
    - Build the app as defined in the source code section
    - Open an FTP Connection to 54.147.196.34
    - Navigate to /var/www/admin/
    - Copy the files to this location
    - Restart the admin portal service (i.e. run command: “sudo systemctl restart admin.service”)

4. **Backend service**

- Update the appsettings.json point to production database(Avana_Live).
- Build each microservice app and publish to the file system(separate folder)
- Open an FTP Connection to 54.147.196.34
- Navigate to /var/www/auth/, var/www/crm,var/www/gateway/,var/www/payments/
- Copy the files to this location for appropriate microservice

- Restart the require service
- (i.e. **run command**:
  Crm: sudo systemctl restart avanacrm.service
  Auth: sudo systemctl restart avanaauth.service
  Payments: sudo systemctl restart avanapayment.service
  Gateway: sudo systemctl restart avanagateway.service
- Restart nginx service (i.e. **run command**: “sudo systemctl restart nginx.service”)

Customer Portal Overview

In the **Customer Portal**, after a new customer completes the signup process, they are redirected to https://customerstaging.myavana.com/Questionaire/start to fill out a questionnaire about their hair health. The data collected from this questionnaire is used to create a personalized **Healthy Hair Care Plan (HHCP)**. This plan provides tailored recommendations for hair care products, treatments, and routines, ensuring that each customer receives the most relevant and effective advice for their hair needs.

The **Customer Portal** of MYAVANA offers users an intuitive and personalized experience centered around managing their hair care and health. Below is a detailed description of the key features visible in the image:

1. **Hair Analysis:**
    - **Digital Hair Analysis** allows customers to analyze their hair's current health, structure, and condition through a digital assessment. The tool may provide insights based on a series of questions or data inputs regarding their hair type, concerns, and habits. This analysis is typically used to create a customized hair care plan.
    - **Hair AI-Instant Analysis** allows users to take a photo of their hair, which is then processed by AI to instantly assess its condition. The AI analyzes factors like texture, health, density, and damage, providing immediate insights and personalized recommendations for products, treatments, and styling tips based on the analysis.
2. **The Healthy Hair Care Plan (HHCP):**

- The Healthy Hair Care Plan (HHCP) section allows users to view personalized recommendations for products, tools, style recipes, and ingredients, which are updated by their stylist or health care professional. These recommendations are tailored to the user's hair type, concerns, and goals,challenges, providing a comprehensive plan to improve hair health. The plan may include specific products to use, styling tips, and ingredient details, all curated by experts to support the user's hair care journey.

3. **Hair Assessment:**

    - Users can take assessments or surveys to evaluate their hair's current condition, identify potential issues, and track progress. The results from the hair assessments are used to inform the overall hair care strategy and help personalize recommendations further.

4. **View Shared HHCP:**

    - The "View Shared HHCP" section allows users to view the Healthy Hair Care Plans (HHCP) of other customers.

5. **Book Consultation(Third party):**

    - The portal enables users to schedule a consultation with hair care professionals or stylists. These consultations could be virtual or in-person, depending on the service provided. During these consultations, users can receive personalized advice, product recommendations, or guidance for addressing specific hair issues.

      https://calendly.com/myavana-hair-consultation/myavana-1-1-hair-consultation?month=2024-12

6. **Get Hair Kit:**

    - Users have the option to obtain a customized hair care kit, which may include a selection of products tailored to their individual needs. The kit could contain hair care items such as shampoos, conditioners, treatments, and styling products. This service ensures users have the right tools to implement their Healthy Hair Care Plan.

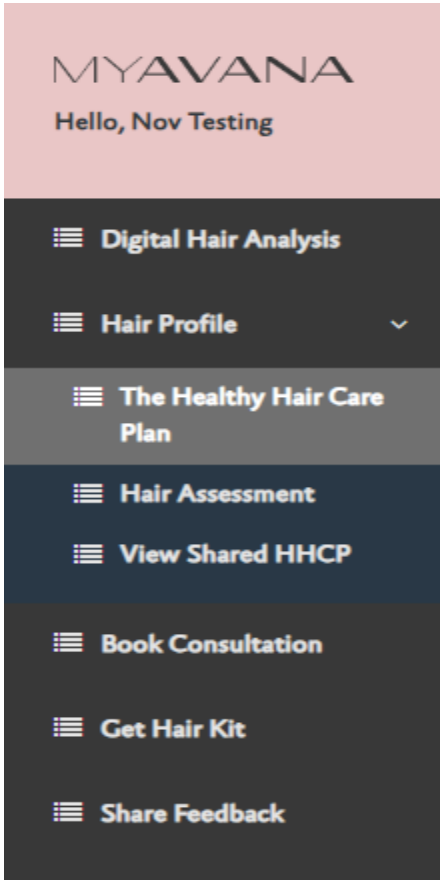
      https://www.myavana.com/pages/scientific-intelligence

7. **Share Feedback:**

    - This feature allows customers to provide feedback about their experience with MYAVANA’s services, products, or consultation. Sharing feedback

helps the platform improve its offerings and maintain customer satisfaction. It may involve ratings, comments, or reviews on specific products or services provided.

https://form.typeform.com/to/IUxMdZaK?typeform-source=customerstaging.myavana.com



High Level Architecture

Please refer to the approach on the retail portal section above to describe the High Level Architecture and how all the microservices connect. Include details of the APi gateway and draw a diagram.

Same as admin portal

Database Design

Please refer to the approach on the retail portal section above to break down the data model along major Entities (Product/HHCP, MYDA etc). Generate database diagrams to show tables used by major entities.

Same as admin portal

Product Recommendation Logic and Auto HHCP Logic

Please refer to the approach on the retail portal section above, discuss the current recommendation logic for products and videos. Provide a break down and point to the where this is implemented (either code file name and function or Stored Procedure)

The customer portal provides an interactive platform for users to answer questions about their preferences and needs. Based on the responses, the system recommends products tailored to their requirements.

Stored Procedures - Same as admin portal

Source Code and Design Information

Similar to the retail portal section above, discuss each microservice, framework, source code location, current branch, architectural peculiarities, and any notes of interest. Include how to build and deploy and what environment variables are needed

Customer Portal Source Information

- **Framework**: .NET Core 2.2
- **Source Location**: https://github.com/MyavanaTechnology/MyavanaAdminV2
- **Latest Branch**: mater
- **Architectural Framework**:
    - The solution is designed around Clean Architecture principles. It connects to the Myavana Service exclusively for all backend requests. It doesn't have any database.
- **Build and Deploy Notes**:
    - Update the appropriate appsettings.json to point to the correct backend service [i.e. either https://apistaging.myavana.com/ (for staging) or https://api.myavana.com/ (for production)]. Generally, if no changes are made to appSettings file, don't copy them as part of the deployment.
    - Publish the code to the file system and then ftp as detailed in the deployment section below
    - This portal has a staging environment as well as production.

Deployment Information

Customer Portal

Staging Environment Deployment

- Ensure that the WebApiBaseUrl in AppSettings.json file is pointing to the correct environment to staging api url(https://apistaging.myavana.com/)
- Build the app as defined in the source code section
- Open an FTP Connection to 52.54.106.156
- Navigate to /var/www/customerstaging/
- Copy the files to this location.
- Restart the customer portal service (i.e. run command: “sudo systemctl restart customerstaging.service”)

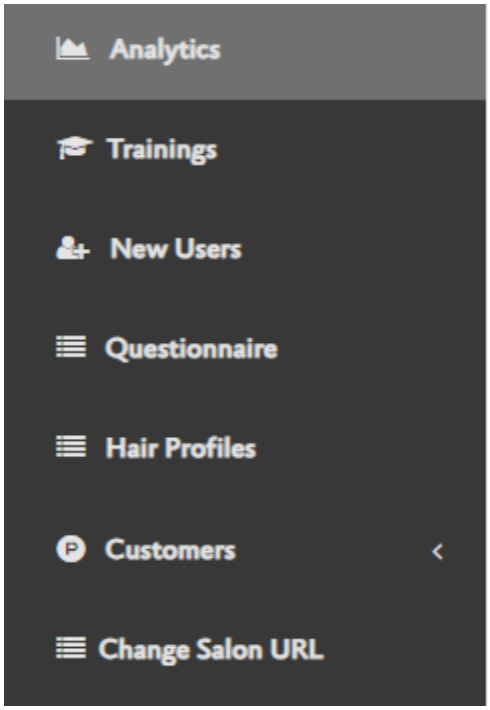
Production Environment Deployment

- Ensure that the WebApiBaseUrl in AppSettings.json file is pointing to the correct environment to staging api url(https://api.myavana.com/)
- Build the app as defined in the source code section
- Open an FTP Connection to 54.147.196.34
- Navigate to /var/www/customer/
- Copy the files to this location.
- Restart the customer portal service (i.e. run command: “sudo systemctl restart customer.service”)

Salon Portal

The Salon Portal provides salon administrators or staff with various tools and features to manage the salon, clients, and services effectively. Here's an overview of the key sections available:

1. **Analytics**: This section provides detailed insights into salon performance, user behavior, and other key metrics. It helps salon administrators track the effectiveness of their services, identify trends, and make data-driven decisions.
2. **Trainings**: This feature provides access to educational resources, training sessions, or webinars that assist salon staff in improving their skills or staying updated with the latest trends, products, or best practices in the industry.
3. **New Users**: This section is used to add users to the salon system.
4. **Questionnaire**: This feature allows salon users to view the surveys or questionnaires filled out by customers.
5. **Hair Profiles:** This section allows salon users to create and manage detailed hair profiles for their clients. Information such as hair type, challenges, care routines, and product preferences is stored here, helping to offer tailored recommendations or treatments.
6. **Customers**: This area allows salon users to add new customers and create or update customer questionnaires.
7. **Change Salon URL**: This section allows salon owners to update the salon URL.



Architectural Overview

Talk about how this is a subset of the Admin portal and detail the customizations specific to this

The Salon Portal is a web-based application designed to streamline salon operations by offering features such as analytics, training management, user onboarding, questionnaire customization, hair profile management, customer records, and salon-specific URL updates. Built with a multi-layered architecture, the portal uses ASP.NET Core for the backend, Razor Pages for the frontend, and SQL Server for data storage. Key functionalities include real-time performance dashboards, training module tracking, dynamic customer feedback forms, and role-based access control (RBAC) for secure user management. Admins can perform CRUD operations on questions, user profiles, and customer data, while customers interact with personalized questionnaires to receive tailored recommendations.

Backend (this should be within the admin portal discuss)

MYAVANA Pro Solution

Admin Portal Pro Overview

The **Admin Portal Pro** combines the features of Myavana's separate portals for Admin, Salon, and Customer into one unified system. This eliminates the need for multiple portals, avoiding duplication, reducing costs, and ensuring a consistent user experience across all platforms, including apps, kiosks, and APIs. By centralizing functions like HHCP, product recommendations, and customer management, and using scalable cloud infrastructure with an API-first approach, it simplifies maintenance, improves reliability, and supports seamless integration for all users.

The **Admin Portal Pro** streamlines Myavana's roles into a unified platform, catering to both Myavana staff and Tenant Partners with clear role-based access. For Myavana staff, **SuperAdmin** has full access to all tenant information and platform functions, while **Admin** has similar capabilities without tenant visibility or SuperAdmin delegation. **Data Entry** focuses on managing product data and attributes but cannot delete. For Tenant Partners, **TenantAdmin** manages tenant-specific data, users, and products, while **TenantUser** supports tenant operations without user or product management. This unified structure simplifies operations, enhances role clarity, and ensures secure, tailored access for each user type.

MYAVANA Users include **DataEntry** (able to see the products), **Admin** (same as now but no tenant switching), and **SuperAdmin** (full platform access with tenant visibility). **Tenant Users** include **Tenant Admins**, who manage tenant-specific products and users, and **Tenant Users**, who can access tenant data but cannot manage products or add users. This structure introduces tenant-specific product management while ensuring secure, role-based access for all users.

MYDA Management System Schema

The MYDA system will use four main tables to manage and track user interactions: **Questions**, **QuestionOptions**, **QuestionSessions**, and **QuestionResponses**. Each table serves a specific role in storing and managing data related to the assessment process. The details for each table are outlined below.

The **Questions** table will store all the questions used in the MYDA. Each question will have a unique Id and include attributes like the question text, type of input (e.g., Radio, Checkbox, Text Area, Textbox), and selection constraints. The MinimumSelection and MaximumSelection columns will determine the range of valid selections a user can make, particularly for checkboxes. Additionally, standard audit columns like CreatedBy, CreatedDate, ModifiedBy, and ModifiedDate will be included to track changes over time.

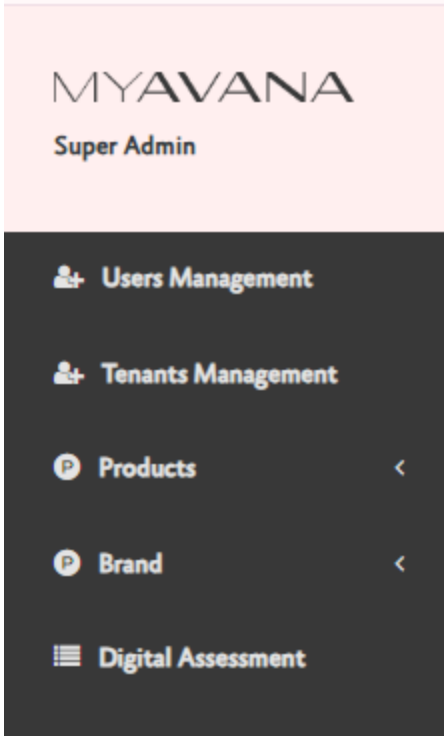
The **QuestionOptions** table will manage the options available for each question. This table will have a QuestionId to link each option to its parent question. Other columns include Option (the display text for the option), Description (optional additional details), Order (to define the display sequence), and an IsDeleted flag to mark options for archival purposes. Like the other tables, it will also include audit fields for tracking creation and modifications.

The **QuestionSessions** table is a new addition to track the different sessions where users engage with the MYDA. Each time a user starts a new assessment (e.g., on an app, kiosk, or customer portal), a new entry will be created in this table. The table will include a unique Id, the UserId of the respondent, the Type of assessment (e.g., HairAI or Full), and a flag (IsComplete) to indicate whether the user completed the session. The table will also include tenant-specific information and standard audit fields.

Questionnaire table will store user responses to the MYDA questions. This table will include a unique Id, the UserId, and references to the QuestionId and SelectedOptionId to link responses to the respective questions and options. For free-text responses, a ResponseText column will be available. Each response will also be linked to a specific session via the SessionId, and tenant-related and audit information will be tracked as well.

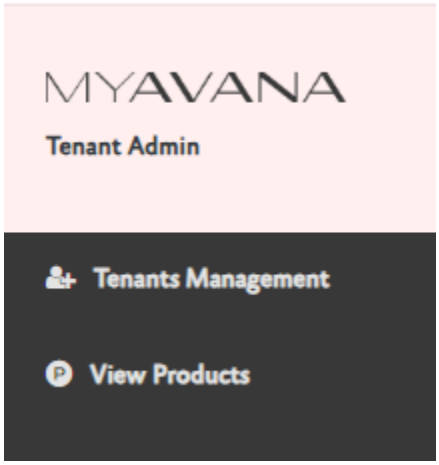
This schema will form the foundation for replicating the current Questionnaire UI, with future plans to allow updates and the addition of new questions directly through the interface. The design ensures a clear structure for storing and managing assessment data while supporting extensibility for future enhancements.

Super Admin



Tenant Admin

The Tenant Admin has the ability to manage users and products within the system. They can add, edit, and delete users, as well as view the list of users associated with the tenant. In addition, the Tenant Admin can view and modify product details, ensuring that product information remains up-to-date.



Myavana Pro High-Level Architecture

Target Framework

The Myavana Pro platform is built using .NET 7.0, leveraging its performance optimizations, modern features, and support for scalable, cloud-native applications.

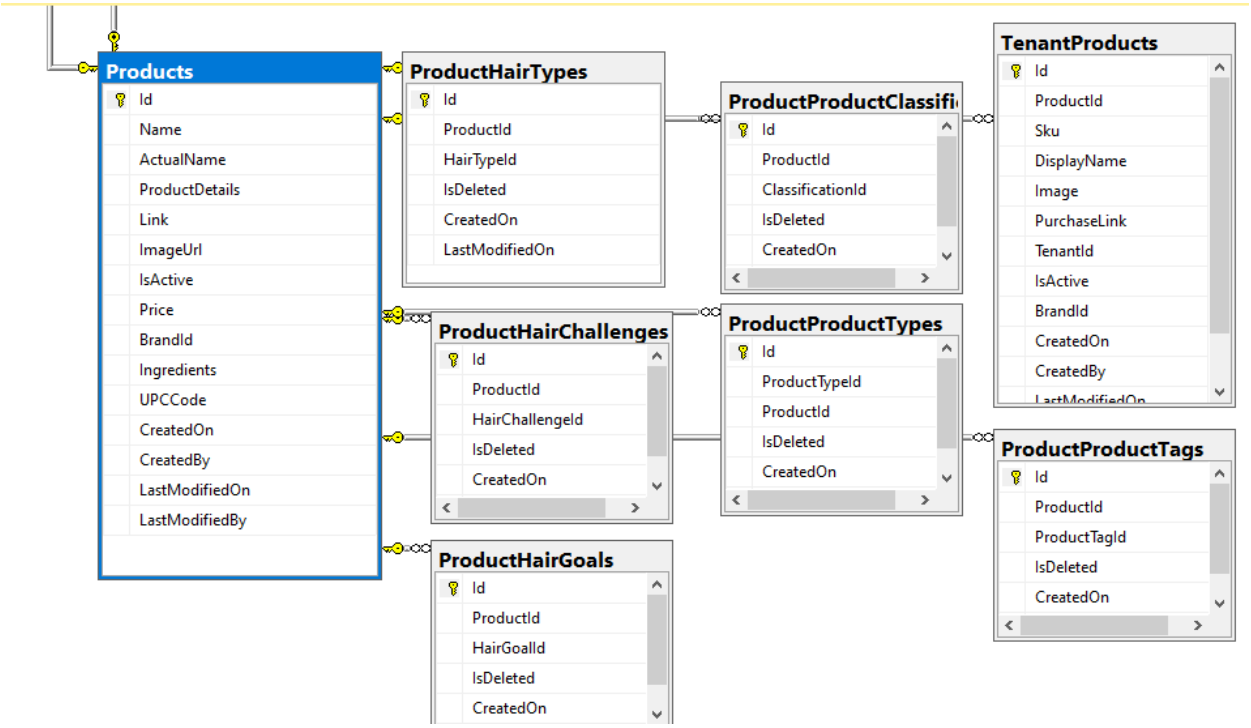
Key Components

1. **Admin Portal**
    - The centralized hub for managing all functionalities and data interactions across the platform.
    - **Responsibilities:**
        - Acts as the primary interface for Myavana staff and tenant users.
        - Communicates with all microservices to provide real-time data and ensure smooth operations.
        - Role-based access ensures secure and tailored user experiences.
2. **Microservices**
   Myavana Pro employs a microservices architecture, ensuring modularity, scalability, and maintainability:
    - **MyAvana.Auth.Api**
        - Handles authentication and authorization for all users.
        - Supports role-based access control (e.g., SuperAdmin, Admin, DataEntry, TenantAdmin, and TenantUser).
        - Ensures secure user authentication with modern protocols like OAuth 2.0 and JWT.
    - **MyAvana.CRM.Api**
        - Manages customer data, including profiles, preferences, and interactions.
        - Enables customer relationship management for both Myavana staff and tenants.
        - Integrates with product recommendation and HHCP functionalities.
    - **MyAvana.Payments.Api**
        - Processes payments for retail transactions, subscriptions, and other financial operations.

Database Diagram

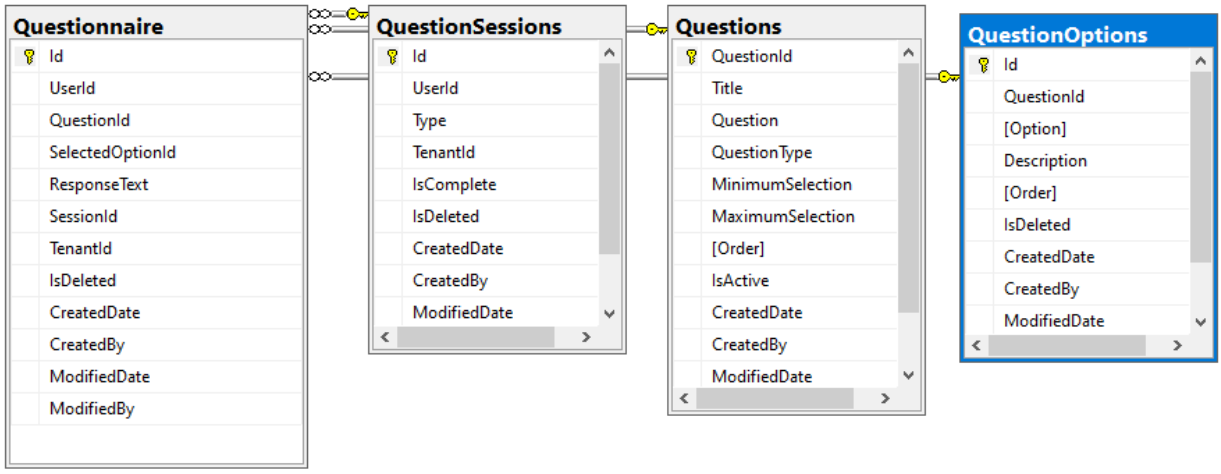
Product table

- dbo.Products
  - Columns
    - Id (PK, FK, int, not null)
    - Name (nvarchar(400), not null)
    - ActualName (nvarchar(400), null)
    - ProductDetails (nvarchar(max), null)
    - Link (nvarchar(1000), null)
    - ImageUrl (nvarchar(1000), null)
    - IsActive (bit, not null)
    - Price (decimal(18,2), null)
    - BrandId (int, null)
    - Ingredients (nvarchar(max), null)
    - UPCCode (nvarchar(max), null)
    - CreatedOn (datetime2(7), not null)
    - CreatedBy (uniqueidentifier, null)
    - LastModifiedOn (datetime2(7), null)
    - LastModifiedBy (uniqueidentifier, null)



Questionnaire table

- dbo.Questionnaire
  - Columns
    - Id (PK, uniqueidentifier, not null)
    - UserId (nvarchar(max), null)
    - QuestionId (FK, int, not null)
    - SelectedOptionId (FK, uniqueidentifier, null)
    - ResponseText (nvarchar(max), null)
    - SessionId (FK, uniqueidentifier, not null)
    - TenantId (uniqueidentifier, null)
    - IsDeleted (bit, null)
    - CreatedDate (datetime2(7), null)
    - CreatedBy (uniqueidentifier, null)
    - ModifiedDate (datetime2(7), null)
    - ModifiedBy (uniqueidentifier, null)

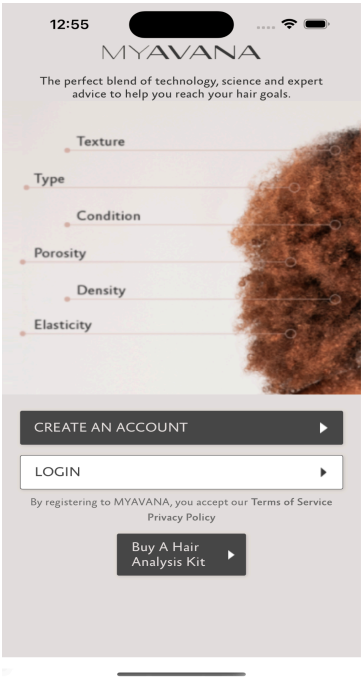


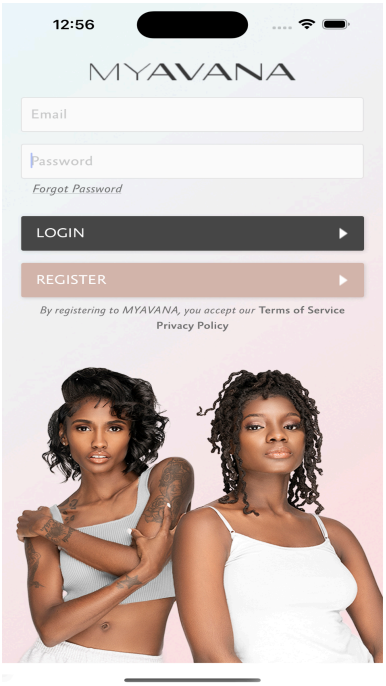
Mobile Applications

MYAVANA AI

UI Design (iOS/Android)

Login Page





Purpose of the Login Page

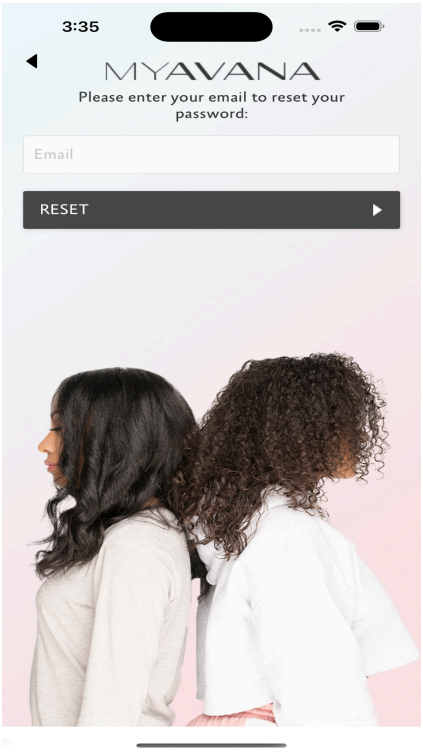
The login page allows registered users to authenticate themselves and gain access to an application. The login process typically requires a user to provide a username (or email) and password. Once the credentials are validated, the user is granted access to their account.

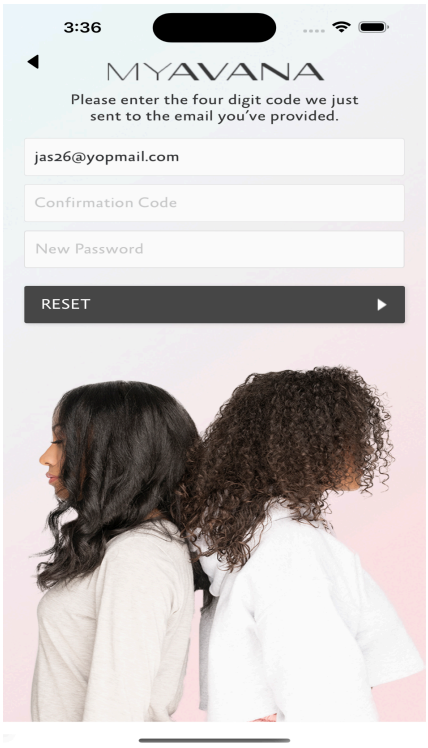
Backend Logic of the Login Page

Authentication Process

1. **User Input**: The user submits the login form containing their username/email and password.
2. **Validation**:
    - The server checks the validity of the username or email in the database.
    - The server compares the entered password with the stored password hash.
3. **Success**: If the credentials are correct:
    - A JWT (JSON Web Token) is generated and sent to the users.
    - The user is redirected to the dashboard or home page.
4. **Failure**: If the credentials are incorrect:
    - Return an error message (e.g., "Invalid credentials") and prompt the user to try again.

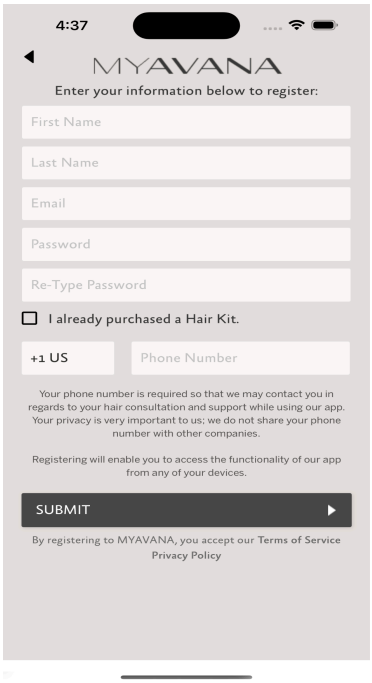
- ## Forgot Password:

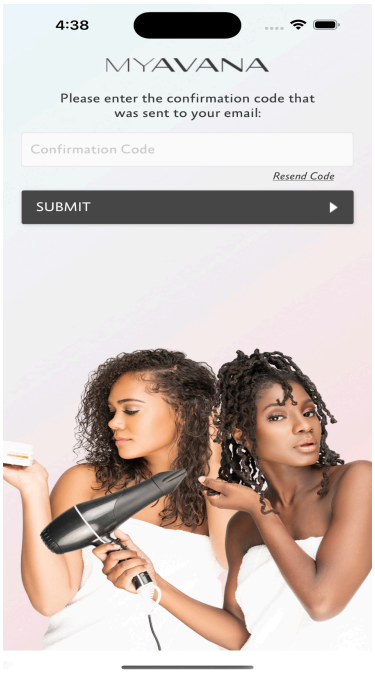




Users can reset their password by entering their email address. A confirmation code will be sent to the provided email. After verifying the code, users can set a new password.

- **Register Page**





Purpose of the Registration Page

The registration page allows new users to create an account on the application by submitting personal information. The page typically requires users to provide details such as a First name, Last name, email address, password, Phone number etc, Once the information is submitted and validated, the user account is created.

Backend Logic of the Registration Page

3.1 User Registration Process

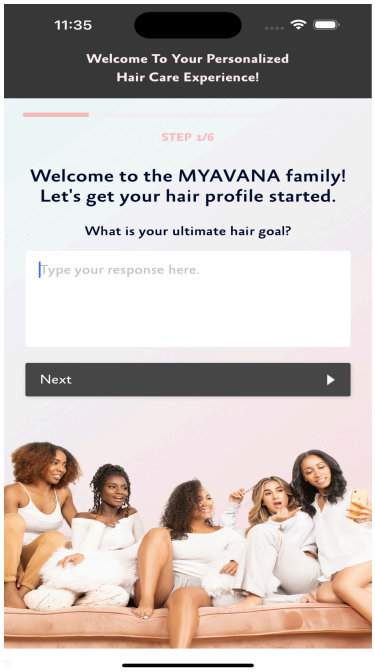
1. **User Input**: The user submits the registration form containing their username, email, password, and any other required information.
2. **Validation**:
    - **First Name**: Ensure that the first name is entered into the text input.

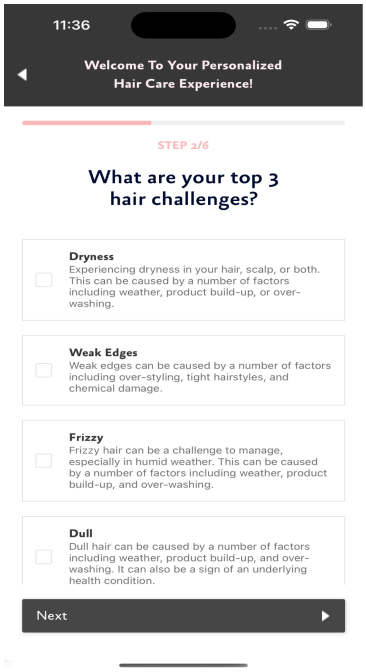
    - **Last Name**: Ensure that the Last name is entered into the text input.
    - **Email**: Validate the email format and check if the email is already registered.
    - **Password**: Ensure the password meets minimum security standards (e.g., minimum length, complexity).
    - **Confirm Password**: Ensure that the "Confirm Password" matches the "Password" field.
3. **Account Creation**:
    - If all inputs are valid, hash the password and store the user's data (e.g., in a database).
    - A confirmation email may be sent to the user for verification.
4. **Success**:
    - After successful registration, the user is redirected to the verification screen to verify the account.
5. **Failure**:
    - If any validation fails (e.g., duplicate email, password mismatch), the backend returns an appropriate error message for display on the front end.

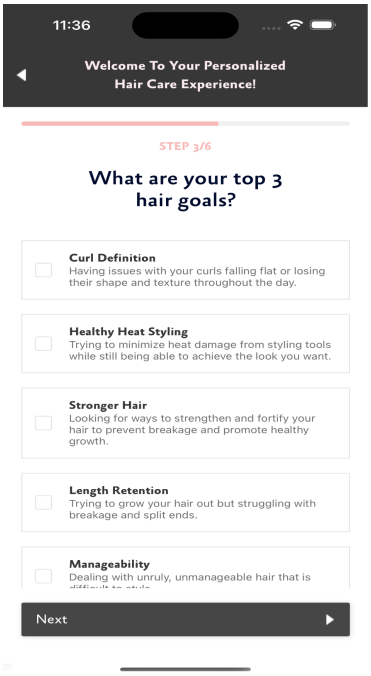
3.2 Account Verification

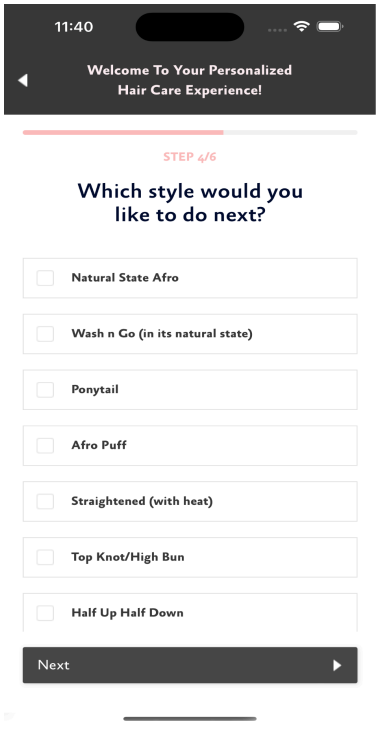
- After the user registers, a verification email can be sent to confirm the email address. The email will typically contain an OTP that the user must fill out to activate their account.

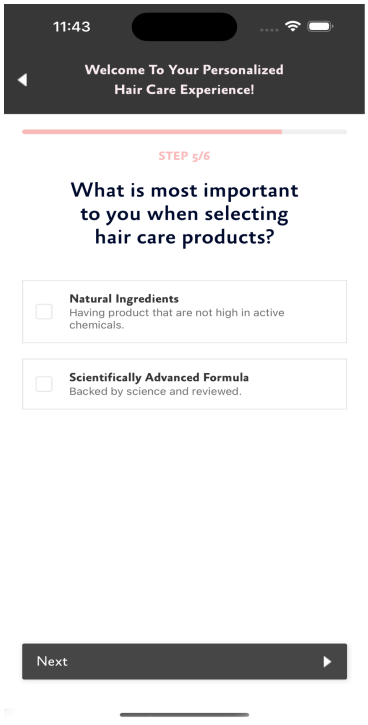
After successfully verifying their account, the user will be redirected to the **Questionnaire** screen, where they can fill out their questionnaire.

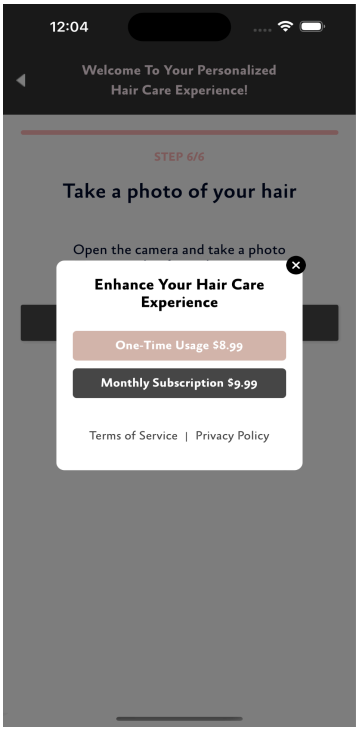












After completing all the steps, users will be prompted to take a photo of their hair. To access this feature, users are required to purchase a subscription.

If users click on the “No” button they will be redirected to the homepage of the app, but they will not be able to use the Hair AI feature until they purchase a subscription plan.

If users click the "Yes" button to buy a subscription, they will see a modal with the subscription options**:**

Subscription Options:

- **One-time usage: $8.99**
  (Users can access this AI feature once for a single use)
- **Monthly subscription: $9.99**
  (Users can access the feature for one month. Subscriptions will auto-renew each month until cancelled.)

Payment Method: In-app purchase(*(Monthly subscriptions include a 7-day free trial version)*

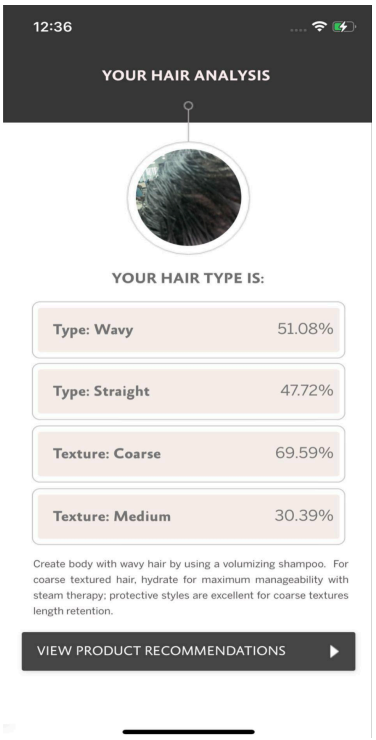
Backend Verifies Purchase with App Store / Play Store

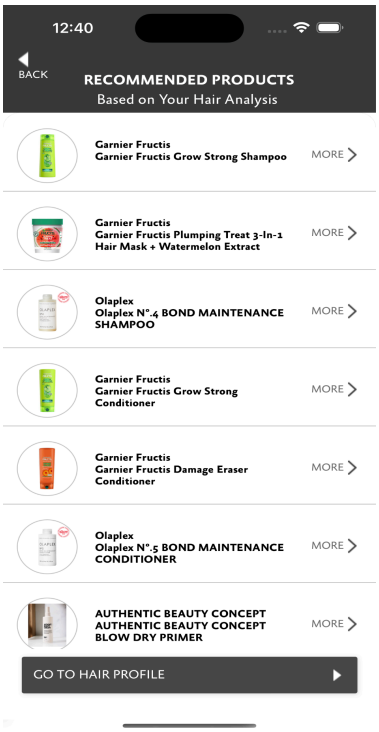
The backend must securely verify the purchase with the respective platform (Apple or Google). This is necessary to confirm that the purchase is valid.

> **For iOS**: Send the receipt to Apple’s **App Store Server** for verification.
>
> **For Android**: Send the purchase token to Google’s **Google Play Developer API** for verification.

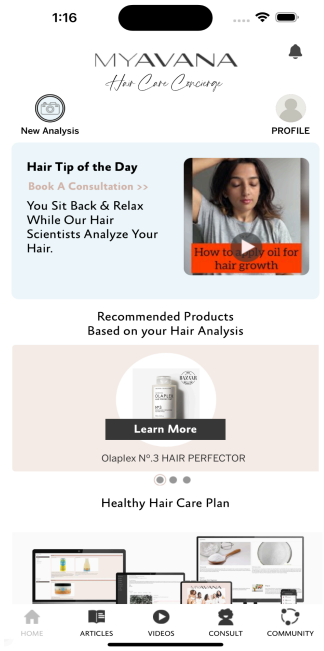
Once the purchase receipt is verified, the user will have access to the Hair AI feature and can view the analysis results after capturing an image of their hair. In this case, the HHCP will be generated.

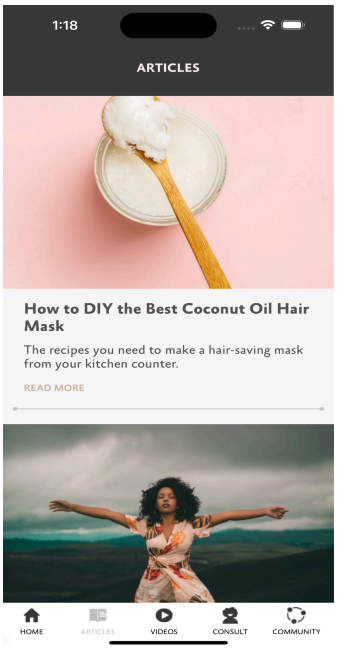
After clicking the '**VIEW PRODUCT RECOMMENDATIONS**' button, the user will see the recommended product based on their Hair Analysis.



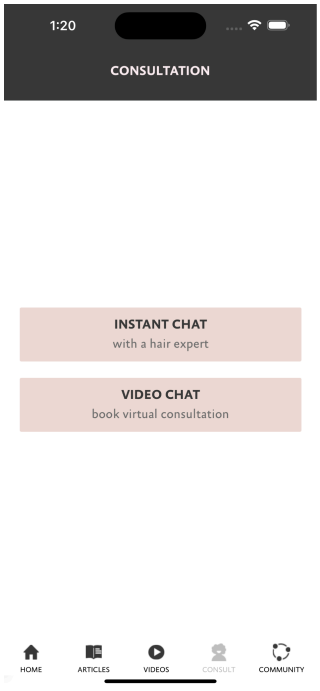


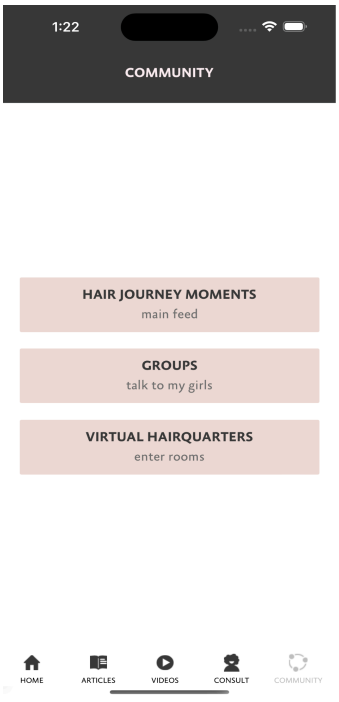
- **Home/Dashboard Page**













The **Home Page** is the default page that opens when the user logs in to the app. The Dashboard has four distinct tabs:

1. Articles Tab

This section displays the articles available within the app.

2. Videos Tab

This section provides a collection of videos, organized by categories.

3. Consult Tab

This section offers two options for consultations:

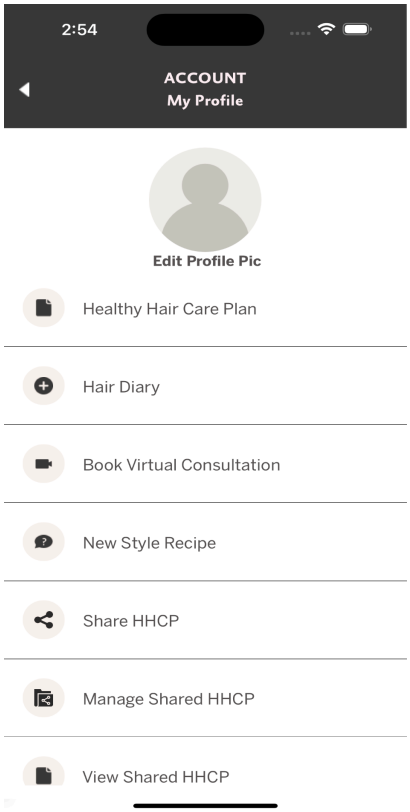
- **Instant Chat**:
  Users can communicate using a third-party tool.
  https://widget.kommunicate.io/chat?appId={appId}&name={name}&email={email}&phone={phone}
- **Video Chat**:
  This option allows users to book a video consultation via a third-party tool. If the salon provides a custom URL for the consultation, it will be used. If not, the default consultation URL is used:
  https://calendly.com/myavana-hair-consultation/myavana-1-1-hair-consultation

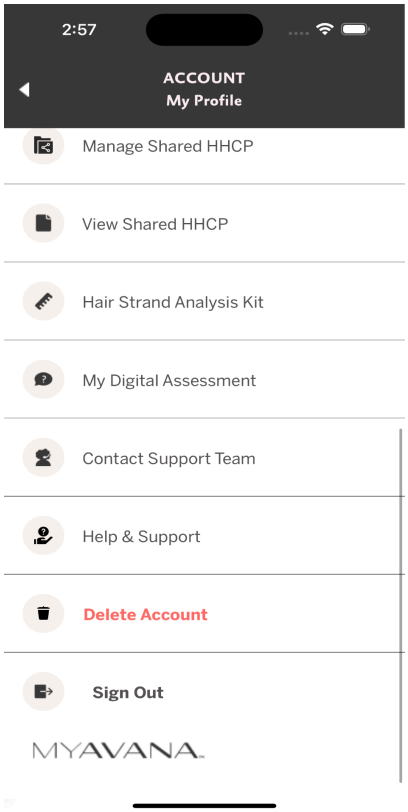
4. Community Tab

This section contains three key areas:

- **Hair Journey Moments**:
  Users can view posts created by other users, sharing their hair journey experiences.
- **Groups**:
  This section defines the various groups available. Users can send requests to join a group. Once approved, they can enter the group and view posts related to that specific group.
- **Virtual Hairquarters**:
  Users can interact in a virtual space using a third-party tool.
  https://labzlive.com/myavana/welcome

- **PROFILE:**





Profile Section

The Profile section contains several options, each with different functionalities:

- **Healthy Hair Care Plan (HHCP)**:
  This section allows users to access their personalized Healthy Hair Care Plan (HHCP) page.
- **Hair Diary**:
  Users can maintain a daily log of their hair care activities and progress in their Hair Diary.
- **Book Virtual Consultation**:
  Users can book a virtual consultation through third-party tools, based on the salon's URL.
- **Share HHCP**:
  Users can share their personalized HHCP with other users.

- **Manage Shared HHCP**:
  This option allows users to manage their shared HHCP, including revoking access to specific users.
- **View Shared HHCP**:
  Users can view the HHCP shared by other users.
- **Hair Strand Analysis Kit**:
  Users can purchase a Hair Strand Analysis Kit via the Myavana website.
- **My Digital Assessment**:
  Users can fill out a digital questionnaire to assess their hair care needs and preferences.
- **Contact Support Team**:
  Users can submit queries or issues by creating a support ticket, allowing them to receive assistance from the support team.

Provisioning Profile and Certificates

Before you can build and distribute the app, we'll need a valid Provisioning Profile and App Signing Certificate.

- **Apple Developer Account:** To create provisioning profiles, you must have an Apple Developer Account.
- **Certificates:** You'll need the following certificates:
  - **Development Certificate**: For debugging and testing on devices.
  - **Distribution Certificate**: For submitting your app to the App Store or TestFlight.
- **Provisioning Profile**: A provisioning profile links the app's build to a set of devices or App Store distribution. You need two kinds of profiles:
  - **Development Provisioning Profile**: Used for testing devices during development.
  - **Distribution Provisioning Profile**: Used for app submission to TestFlight or the App Store.

Steps to Create Provisioning Profiles:

1. **Go to the Apple Developer Portal:**
   - Sign in to Apple Developer and navigate to the Certificates, Identifiers & Profiles section.
2. **Create a New Certificate**:
   - Under Certificates, click the plus (+) button to create a new certificate.
   - Choose either a Development or Distribution certificate.

3. **Create an App ID:**
    - In Identifiers, register your app's App ID (usually based on your app's bundle identifier).
4. **Create Provisioning Profiles:**
    - In Profiles, click the plus (+) button to create a new provisioning profile.
    - Select either Development or Distribution based on the stage (testing or App Store submission).
    - Choose the app ID you created earlier and associate the profile with the appropriate certificate(s).
5. **Download the Profile:**
    - Download the provisioning profile and double-click to install it into Xcode.

Build App in Xcode

Now that your provisioning profile and certificates are set up, follow these steps:

1. **Set the Correct Signing in Xcode:**
    - In Xcode, open your project and navigate to the Signing & Capabilities tab.
    - Select your Team and ensure the correct Provisioning Profile and Certificate are selected for the Debug and Release build configurations.
2. **Build and Archive Your App:**
    - Select Generic iOS Device or a real device as the target.
    - Go to Product > Archive to create an archive of your app.
    - If the build is successful, the Organizer window will appear with the archive.

TestFlight: Upload and Manage Staging/Production Environments

TestFlight allows you to distribute your app to testers. You'll need separate builds for **staging** and **production** environments to differentiate between versions that are still being tested and the final version that's intended for public release.

Steps to Upload to TestFlight:

1. **Log in to App Store Connect**:
    - Go to App Store Connect, sign in with your Apple Developer account, and select your app.
2. **Create App Versions**:
    - Under the **App Store Connect > My Apps** section, create a version for both staging and production if you haven't already.

    - For the staging version, use a different **build number** (e.g., `1.0.0`).
3. **Upload the Build to App Store Connect**:
    - In **Xcode**, open the Organizer window and select your archive.
    - Click **Distribute App** > **App Store Connect** > **Upload**.
    - Xcode will automatically upload your build to App Store Connect.
4. **Enable TestFlight Testing**:
    - Once the build is uploaded, it will appear in the **TestFlight** tab of App Store Connect.
    - For **Staging**: Select the build and set it for **Internal Testing** (team members with access to the app's App Store Connect account).
    - For **Production**: Set the build for **External Testing** and invite testers by email or through a public link.
5. **Testers Access**:
    - Internal testers (up to 25 members) can access the app immediately.
    - External testers (public or private) need to be approved by Apple, which typically takes 24–48 hours.
6. **Managing Staging/Production Builds**:
    - Keep your staging builds separate from production by versioning your builds properly (e.g., `1.0.0-staging` for staging and `1.0.0` for production).
    - You can have different configurations (API endpoints, database URLs, etc.) in the app to handle staging and production environments.

Releasing to the App Store

Once your app has passed testing in TestFlight:

1. **Submit for Review**:
    - In-App **Store Connect**, go to the **App Store** tab and submit your app for review.
    - Fill out all necessary metadata (description, keywords, screenshots, etc.), and select the build you want to release.
2. **App Review**:
    - Apple will review your app, which usually takes 1–3 business days, though it can take longer if there are issues or if it's a major update.
3. **Release**:
    - Once approved, you can release your app to the App Store immediately or schedule the release.

Android App

Handling Staging and Production Environments

Android apps can be configured to support different environments (staging and production) by managing **build variants** and using **Gradle** build configurations.

a. Using Build Variants for Staging and Production:

In Android Studio, you can configure **build types** and **product flavors** to manage different environments.

1. **Build Types**:
    - By default, Android provides two build types: `debug` and `release`. You can customize these types for different environments.
2. **Product Flavors**:
    - Product flavours allow you to create different versions of your app. You can create separate configurations for **staging** and **production**.

Example `build.gradle` setup:

```
android {
    compileSdkVersion 34
    defaultConfig {
        applicationId "com.example.myapp"
        minSdkVersion 21
        targetSdkVersion 34
        versionCode 1
        versionName "1.0"
    }

    buildTypes {
        debug {
            buildConfigField "String", "BASE_URL",
"\"https://staging-api.example.com/\""
        }
        release {
```

```
            buildConfigField "String", "BASE_URL",
"\"https://api.example.com/\""
            // Other optimizations for release (minify, etc.)
        }
    }

    productFlavors {
        staging {
            applicationIdSuffix ".staging"
            versionNameSuffix "-staging"
        }
      }
}
```

- **`staging` Flavor**: Used for development, testing, and QA purposes. It may connect to a staging server, and the app's package name might include a `.staging` suffix.
- **`production` Flavor**: Used for the final production version. No suffix in the package name, and it connects to the live API.

By configuring **proflavoursavors** and **build types**, you can easily create and test different builds (staging vs. production) of your app.

App Signing Configuration

For Android, app signing is a critical part of the build process. You'll need a **keystore** to sign your app for both staging and production environments.

a. Creating a Keystore:

Open a terminal and run the following command to create a keystore (for the release version):

```
keytool -genkeypair -v -keystore my-release-key.keystore -keyalg
RSA -keysize 2048 -validity 10000 -alias my-key-alias
```

1. Replace `my-release-key.keystore` with the name of your keystore, and `my-key-alias` with the alias of your key. The Keystore will be used to sign your production APKs.
2. Store the keystore file in a secure location and never expose it to the public.

b. Configuring Gradle for App Signing:

You need to configure Gradle to use the Keystore for signing your APKs:

- **For Staging**: You might not need a secure keystore for staging builds, or you can use a different keystore.
- **For Production**: You must use the production keystore to sign the app.

Modify the `build.gradle` to specify the signing configuration:

```
android {
    signingConfigs {
        release {
            storeFile file("path/to/my-release-key.keystore")
            storePassword "keystore-password"
            keyAlias "my-key-alias"
            keyPassword "key-password"
        }

        staging {
            storeFile file("path/to/my-staging-key.keystore")
            storePassword "keystore-password"
            keyAlias "my-key-alias"
            keyPassword "key-password"
        }
    }

    buildTypes {
        release {
            signingConfig signingConfigs.release
        }
        debug {
```

```
            signingConfig signingConfigs.staging
        }
    }
}
```

- For the **release** build type, specify the production keystore.
- For **staging**, you can either use the same keystore with a different alias or generate a new keystore specifically for staging.

Building/Generate APKs for Staging and Production

Once the environment is configured, you can build the APKs for staging and production.

a. Building APK for Staging:

1. **In Android Studio**, select the **Build Variant** for **staging** (from the Build Variants panel).
2. Then, go to **Build > Build Bundle / APK > Build APK** to generate the APK.
3. This will produce an APK with the `-staging` suffix or as defined in your build configuration.

b. Building APK for Production:

1. Select the **release** variant from the **Build Variants** panel in Android Studio.
2. Go to **Build > Build Bundle / APK > Build APK** to generate the production APK.
3. Ensure the app is signed with the production keystore (configured in the Gradle file).

You can also build the APKs using the command line:

For staging APK:
`./gradlew assembleStagingRelease`

- For production APK:
  bash
  Copy code
  `./gradlew assembleRelease`

- These commands will create signed APKs that you can upload to distribution platforms like **Google Play** or use for testing.

Distributing APKs

Once the APKs are built, you can distribute them to testers or submit them to the **Google Play Store** for production.

Testing APKs (Manual or other services):

- For **staging versions**, you can distribute the APK manually via email, or use services like **Diwai** or **TestFairy** for easy distribution to testers.

Uploading to Google Play (Production)

Uploading an Android app to the **Google Play Console** involves several steps, from preparing the app bundle (AAB), filling in app details, and submitting the app for review. Below is a step-by-step guide for uploading your Android app to the **Google Play Console**.

Step 1: Prepare Your Android App

Before uploading, ensure your app is ready for release.

1. **Generate a Signed App Bundle (AAB) or APK**:
    - Google Play Console now recommends using **App Bundles (AAB)** over APKs for optimized app delivery. Ensure your app is signed correctly for production.
    - **App Bundle** (.aab) is the preferred format, and the APK is still supported but is being phased out in favour of App Bundles.

2. **Check Version Code and Version Name**:
    - Open `build.gradle` and ensure the **versionCode** and **versionName** are updated. Google Play uses **versionCode** to identify new versions of your app, so each new version must have a higher version code.
    -

```
defaultConfig {

applicationId "com.example.myapp"
```

```
    minSdkVersion 21

    targetSdkVersion 33

    versionCode 2  // Increment this for every new release

    versionName "1.1"  // Update version name accordingly

}
```

3. **Create a Keystore** for signing (if you haven't already) to sign your app. This ensures that only you can update your app in the future.

Step 2: Sign Your App

For **production** releases, the app needs to be **signed** with a release key.

1. **Sign the App** using your release keystore.
2. In **Android Studio**, use **Build > Build Bundle(s) / APK(s) > Build Bundle** to generate a signed **App Bundle (.aab)**.
3. The generated **AAB** file will be located in the `app/build/outputs/bundle/release/` directory.

If you are using **Gradle** to build the app, you can use the following command to build the AAB:

```
./gradlew bundleRelease
```

Step 3: Sign In to Google Play Console

1. **Go to Google Play Console**:
    - Navigate to Google Play Console.
    - Sign in with your **Google Developer Account** (or create one if you don't have one).
2. **Create a New App**:
    - If this is your first time submitting an app, click **"Create Application"**.
    - Select the **default language** and enter the **title** of your app.
3. **Fill Out the App's Metadata**:
    - In the **"Store Listing"** section, provide detailed information such as:
        - **Description**: What your app does and why it's useful.

        - **Screenshots**: Upload screenshots of your app for various screen sizes.
        - **App Category**: Choose the correct category for your app (e.g., Game, Productivity).
        - **Contact Information**: Email, website, and phone number.
        - **Privacy Policy**: A link to your app's privacy policy, if necessary.
4. **Prepare for Release**:
    - Provide information such as **Content Rating**, **Pricing & Distribution**, and **Countries** where your app will be available.
    - Select whether the app is **free** or **paid**.

Step 4: Upload the App Bundle (AAB) to Google Play Console

1. **Navigate to the "Release" Section**:
    - Go to your app's page in **Google Play Console**.
    - Under the **"Release"** section, click on **"Production"** (or you can use **Beta** or **Internal Testing** if you're testing the app).
2. **Create a New Release**:
    - Click on **"Create Release"**.
    - You'll be prompted to upload your **App Bundle (AAB)** file.
3. **Upload the AAB File**:
    - Click the **"Upload"** button and select the **AAB** file you generated earlier (`app-release.aab`).
    - After the upload is complete, Google Play Console will validate the file and show you the details of the new release.

Step 5: Add Release Notes

1. **Release Notes**:
    - Provide release notes for your users. This is where you'll highlight the changes in the new version of the app (e.g., bug fixes, new features, improvements).
2. **Review**:
    - After filling out the release notes, click **"Review"** to proceed to the next step.

Step 6: Configure Content Rating

1. **Content Rating**:

- You need to complete a **content rating questionnaire** to help Google categorize your app. This will ensure your app is listed in the right category and is shown to the appropriate audience.

2. **Privacy Policy**:
   - If your app collects personal data, you need to link to your **Privacy Policy**.

Step 7: Set Pricing and Distribution

1. **Pricing & Distribution**:
   - Choose whether your app is **free** or **paid**. Once an app is marked as **paid**, it can't be changed to free later, so choose carefully.
2. **Select Countries**:
   - Select which countries your app will be available in.
   - You can also select if the app should be available to specific devices (e.g., Android TV, Wear OS).

Step 8: Submit for Review

1. **Submit for Review**:
   - After reviewing all the details and ensuring everything is correct, click **"Start rollout to production"**.
   - Your app will be sent for review. Google Play will evaluate your app, which typically takes 1–3 business days.
2. **Wait for Approval**:
   - Google will check your app for compliance with **Google Play Policies**.
   - If everything is in order, the app will be **approved** and will go live in the Play Store.
   - If your app is rejected, you will be notified of the reasons, and you can address them before re-submitting.

Step 9: Monitor the Status of Your App

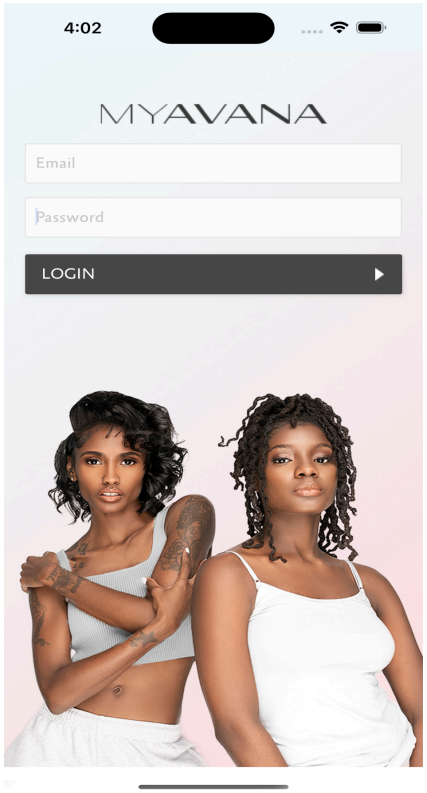
1. **Monitor Your App's Status**:
   - After submission, monitor the **release** status in the Google Play Console. If it's approved, your app will be available on the Play Store.

- If there are any issues or rejections, Google Play Console will provide feedback on the issues so that you can resolve them.

2. **App Analytics**:
   - Use **Google Play Console** to monitor your app's **performance**, including:
     - **Downloads and installs**
     - **Crash reports**
     - **User reviews and ratings**
     - **Financial data** (if app is paid)

MYAVANA PRO

UI Design(iOS/Android)



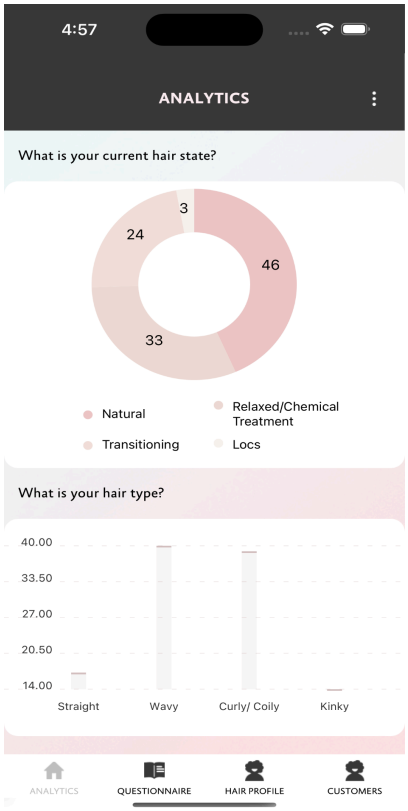
Login Page

- **Salon Admin Login**:
  Only the **Salon Admin** is authorized to log in directly from the login page. The Salon Admin will use their credentials (e.g., email and password) to access the app.

- **Salon Users Creation**:
  Salon Users will be generated by the **Myavana Admin**.

  This means only the admin can create accounts for salon staff, who will then use those accounts to access the app as required.

    - **Analytics**



Analytics Tab

The **Analytics Tab** is the default tab in the **Myavana Pro** app. It provides detailed analytics and insights for the **Salon User**, helping them track performance, customer data, and other key metrics.

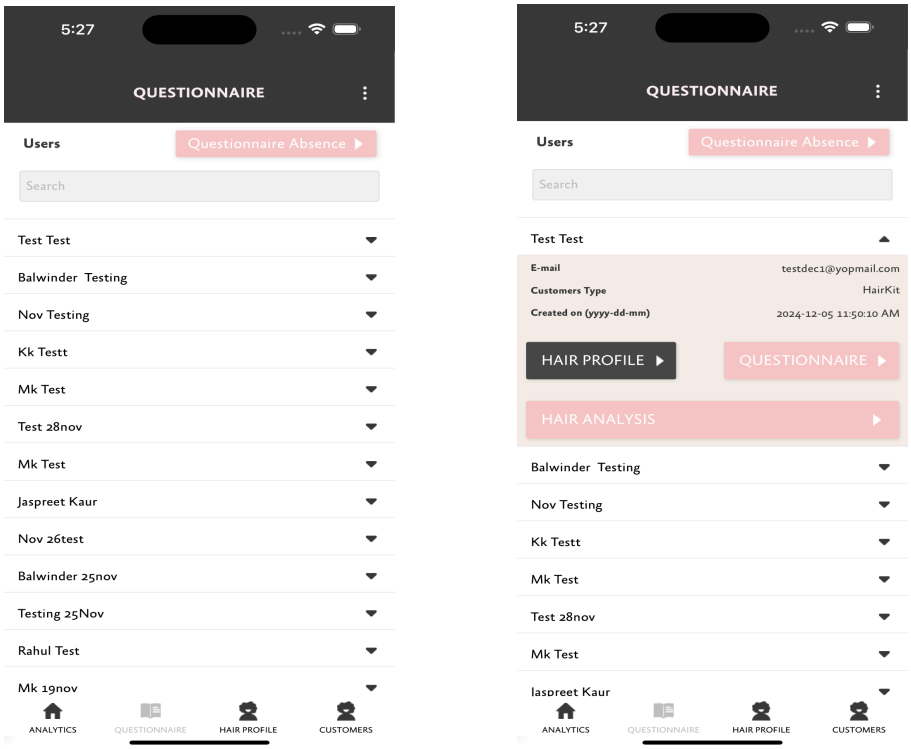
Other Available Tabs:

- **Questionnaire Tab**:
  This tab shows a list of salon users.
- **Hair Profile Tab**:
  This tab shows a list of salon users.
- **Customers Tab**:
  This tab shows a list of salon users.

Questionnaire Tab: This tab shows a **list of salon users**.
The **Salon Admin** can:

1. **Create Hair Profiles** for each user.
2. **Fill Out the Questionnaire** by clicking on the **Questionnaire Absence** button.

3. After the questionnaire is filled out, the Salon Admin can view the completed responses by clicking on the **Questionnaire** button.



Hair Analysis Button

When the **Salon Admin** clicks on the **Hair Analysis** button, a prompt will appear requesting them to complete the **Questionnaire** if it has not been filled out yet. Once the questionnaire is completed, the admin can proceed to access the **Hair Analysis**.

- After capturing an image of the client's hair, the **Salon Admin** can view the **Hair Analysis** results.
- Based on the analysis, the **Healthy Hair Care Plan (HHCP)** will be generated once the **Recommended Products** button is clicked.

Re-run Hair AI Tool

- The **Salon User** has the option to **re-run the Hair AI tool** anytime after the initial analysis by simply clicking the **Hair Analysis** button again. This allows them to update or refine the analysis if needed.

5:44

TEST TEST
YOUR HAIR ANALYSIS



YOUR HAIR TYPE IS:

Type: Straight	99.98%
Type: Coily	0.02%
Texture: Fine	99.74%
Texture: Coarse	0.23%

Bulk up straight hair with a protein shampoo and conditioner. For fine textured hair, defrizz with smoothing shampoos and conditioners, deep treatments help with style memory. Bulk up straight fine hair with a protein shampoo and conditioner.Medium Texture - For medium textured hair, balance with moisture and protein conditioners.

RECOMMENDED PRODUCTS

5:45

TEST TEST

Uploaded Photo of Your Hair

Upload



Test Test

Hair id:

Hi Test !
Your Digital Hair Profile includes your introductory product recommendations based on a quick analysis of your hair. We incorporated your Hair Goals, Hair Challenges, Product Recommendations, and % breakdown of your Unique Hair Type Combination. To get a comprehensive healthy hair care plan, make sure to get your hair analysis kit in the menu to your left. This should get you started in the meantime! If you have any questions, please email us at support@myavana.com
Love,
MYAVANA

Stylist Notes

Add Notes

5:45

TEST TEST

Recommended for you

Products | Ingredients | Regimens | Style Recipes | Scope Analysis

Essential Products

See All

Shampoo



MIXED CHICKS
MIXED CHICKS SULFATE FREE COLOR SAFE SHAMPOO
MORE



Garnier Fructis
Garnier Fructis Sleek & Shine Glass Hair Water
MORE



Garnier Fructis
Garnier Fructis Sleek & Shine Shampoo
MORE



Virtue
Virtue SMOOTH SHAMPOO
MORE

5:56

TEST TEST
YOUR HAIR ANALYSIS



YOUR HAIR TYPE IS:

Type: Straight	99.98%
Type: Coily	0.02%
Texture: Fine	99.74%
Texture: Coarse	0.23%

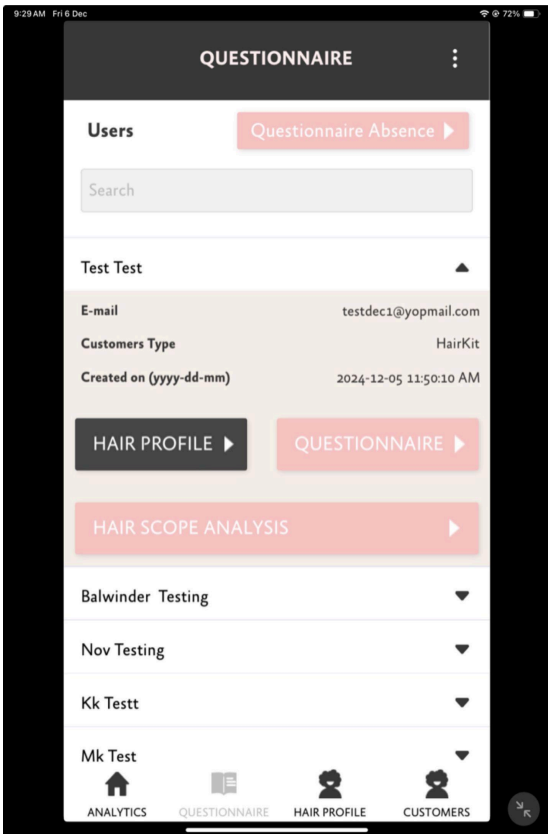
Bulk up straight hair with a protein shampoo and conditioner. For fine textured hair, defrizz with smoothing shampoos and conditioners, deep treatments help with style memory. Bulk up straight fine hair with a protein shampoo and conditioner.Medium Texture - For medium textured hair, balance with moisture and protein conditioners.

RE-RUN

RECOMMENDED PRODUCTS

We have also added additional HairScope functionality to the Myavana Pro app. A new condition has been added to the device. If the salon admin runs the app on an iPad or tablet, they will be able to access HairScope by following these steps:

1. Go to the **Questionnaire** tab.
2. Select the user.
3. Tap on **HairScope Analysis**.
4. On the HairAnalysis screen, select the **HairScope Analysis**.

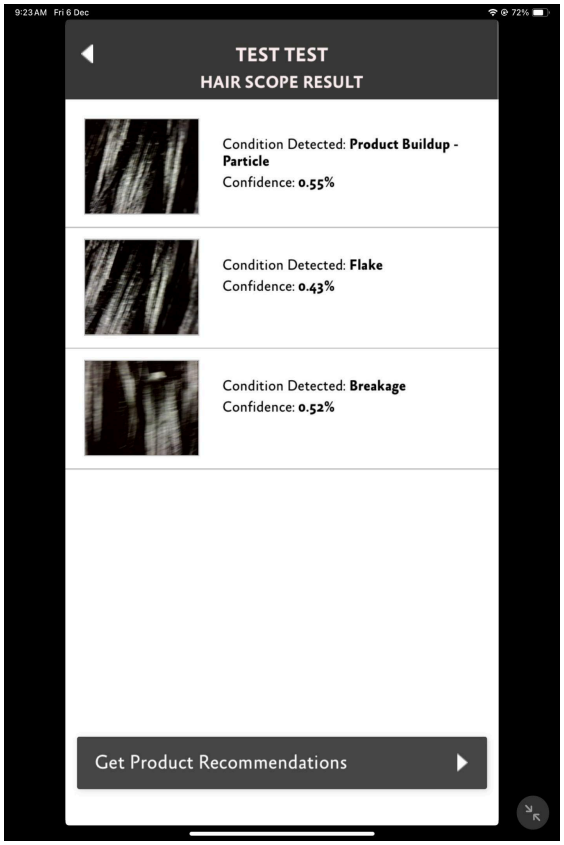


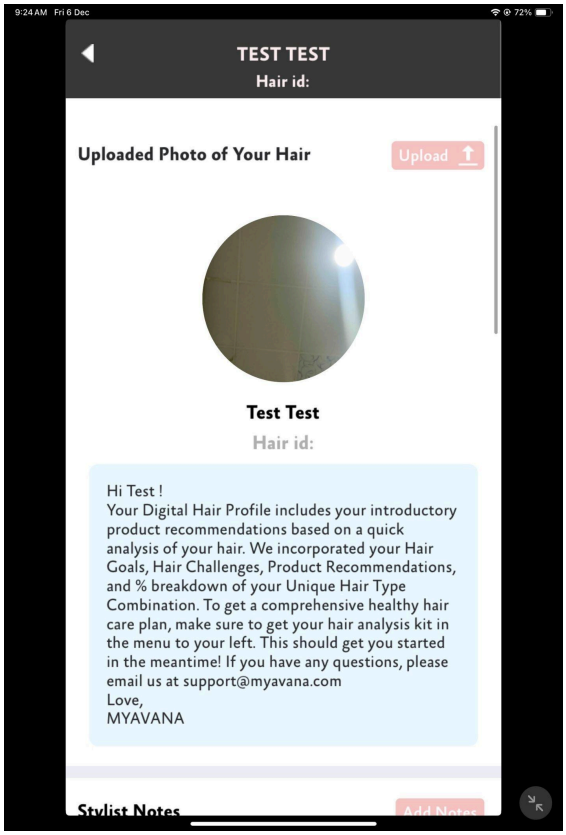


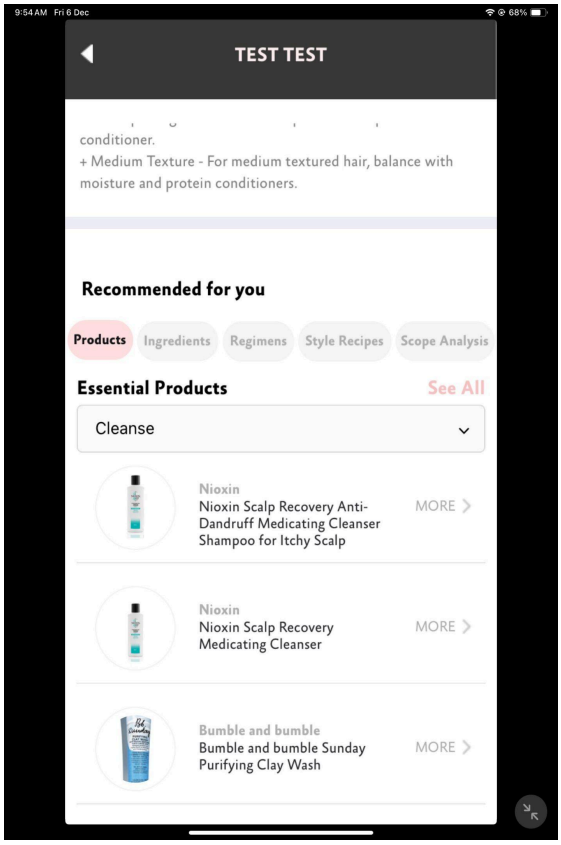
The salon admin needs to attach the HairScope to detect the hair detection. After detection, the salon admin should click the **POST** button at the bottom of the page. Once clicked, the detection results will be displayed on the screen.

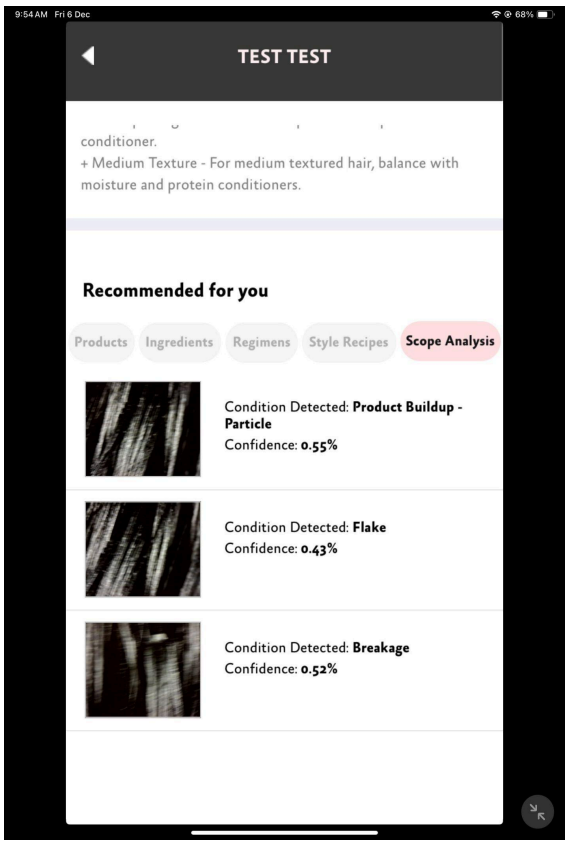
On the HairScope result screen, if the salon admin clicks the **GET PRODUCT RECOMMENDATIONS** button, a user HHCP will be created. Based on the hair detection, product will be generated and displayed on the **Hair Profile** screen.The salon

admin can also view the user's Hair Profile by clicking on the Hair Profile tab and selecting the user. This will display the user's Hair Profile.





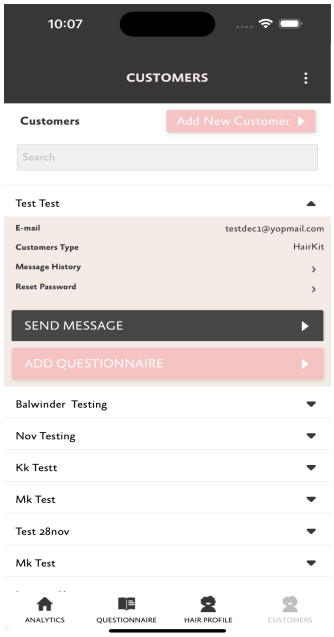


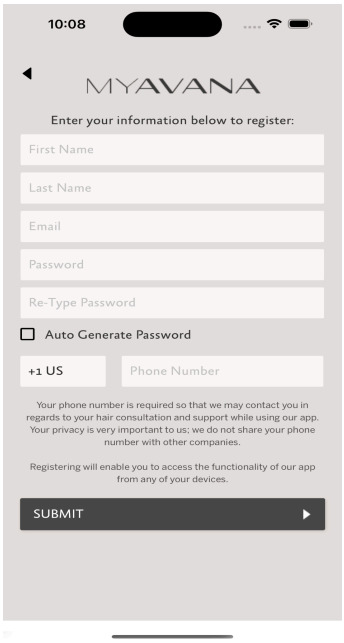


Customers Tab:- This tab shows a **list of salon users**.

The salon admin can also add a new customer/user by clicking on the **Add New Customer** button on the customer screen.

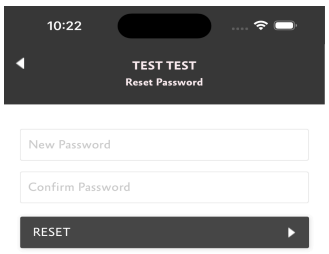
After filling out the form, a new customer will be created and displayed in the Questionnaire, Hair Profile, and Customers tabs.

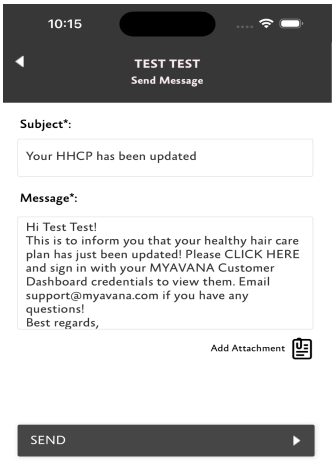


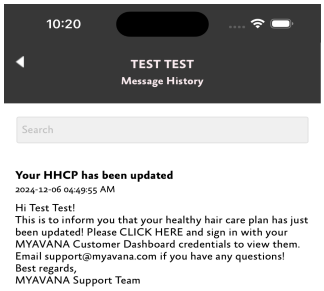


The salon admin can also send a message to the customer by clicking on the **Send Message** button. Additionally, the admin can add a customer's questionnaire by clicking on the **Add Questionnaire** button and filling out the questionnaire.

The admin can reset the customer's password by clicking on the **Reset Password** button. Furthermore, the admin can view the message history by clicking on the **Message History** button.







Provisioning Profile and Certificates

Follow the same steps as above in the **Myavana AI** app. Here is the link

Build App in Xcode

Follow the same steps as above in the **Myavana AI** app. Here is the link

TestFlight: Upload and Manage Staging/Production Environments

Follow the same steps as above in the **Myavana AI** app. Here is the link

Android App

App Signing Configuration

Follow the same steps as above in the **Myavana AI** app. Here is the link

Building/Generate APKs for Staging and Production

Follow the same steps as above in the **Myavana AI** app. Here is the link

Distributing APKs and Uploading to Google Play (Production)

Follow the same steps as above in the **Myavana AI** app. Here is the link

Instant Chat:
Users can communicate using a third-party tool.
https://widget.kommunicate.io/chat?appId={appId}&name={name}&email={email}&phone={phone}

Book a consultation:

Allows users to book a video consultation via a third-party tool. If the salon provides a custom URL for the consultation, it will be used. If not, the default consultation URL is used:

Default url:
https://calendly.com/myavana-hair-consultation/myavana-1-1-hair-consultation

Virtual Hairquarters:

Users can interact in a virtual hairquarters using a third-party tool.
https://labzlive.com/myavana/welcome

Third Party Tools

This section summarizes all the third-party tools we use as part of the MYAVANA suite of products. The credentials to access the services below which don't rely on personal credentials are 1Password (i.e .services using shared passwords)

S/N	Provider Name	Description
1	AWS Cloud	**Login**: https://332244720589.signin.aws.amazon.com/console AWS Cloud is used to manage our hosted servers and databases to support all our services. More details on each infrastructure is provided in the Cloud Infrastructure section below.
2	Replit	**Login**: https://replit.com/@myavana/MyAvana Replit is used to host the production instance of the HairScope Analysis web portal. The staging instance is hosted with AWS Staging Server..
3	GitHub	**Login**: https://github.com/login GitHub is used as our source control repository. It holds the source code of every application we own except the hairScope which is currently managed in Replit Requires personal accounts which Candace can add.
4	Azure DevOps	**Login**: https://dev.azure.com/myavanatechnology/ Azure DevOps is used for managing our product roadmap. It's used in conjunction with Slack to manage the development team process. The most recent project board is available at: https://dev.azure.com/myavanatechnology/General%20Road%20Map/_boards/board/t/General%20Road%20Map%20Team/Issues Requires personal accounts which Candace can add.
5	SendGrid	**Login**: https://login.sendgrid.com/login/identifier

		SendGrid is used as the email sending transportation mechanism across all our products.
6	Cloudfare	**Login**: https://dash.cloudflare.com/login CloudFlare is used for managing all the DNS records for the MYAVANA domain.
7	Stripe	**Login**: https://dashboard.stripe.com/login Stripe is used as our payment processing platform. It's rarely used directly at this point as we've moved payments on the Mobile Apps to In-App subscription and web payments to Shopify.
8	Shopify	**Login**: https://www.shopify.com/login?ui_locales=en Shopify is used as our ecommerce website as well as hosting myavana.com (our website)
9	Hubspot	**Login**: https://app.hubspot.com/login Hubspot is used as our customer relationship management and marketing platform. It's currently integrated to Klaviyo
10	GoDaddy	**Login**: https://sso.godaddy.com/?realm=idp&app=cart&path=%2Fcheckoutapi%2Fv1%2Fredirects%2Flogin GoDaddy is used for managing our domain name.
11	Apple Developer	**Login**: https://developer.apple.com/ Used to manage our iOS Apps testing and deployment to the cloud Requires personal accounts which Candace can add.
12	Google Developer	**Login**: https://console.cloud.google.com/ Used to manage our Android Apps testing and deployment to the cloud Requires personal accounts which Candace can add.
13	ZenDesk	**Login**: https://www.zendesk.com/login/ Used for managing our customer support and ticketing process

14	Twilio	**Login**: https://www.twilio.com/login Twilio is used as our texting platform
15	Zapier	**Login**: https://zapier.com/app/login Zapier is used for automating our workflows.
16	Labz	**Login**: https://labzlive.com/signin Labz is our online web tool powering the MYAVANA Metaverse
17	Gorgias	**Login**: https://www.gorgias.com/login Gorgias was used for customer support in the past.
18	Klaviyo	**Login**: https://www.klaviyo.com/login Klaviyo is used for managing our communication with our customers.
19	FiveTran	**Login**: https://fivetran.com/login?next=%2Fdashboard FiveTran is used as our data injection platform for aggregating data from disparate systems for our analytics purposes.
20	Tableau	**Login**: https://www.tableau.com/tableau-login-hub Tableau is used for generating our BI reports

AWS Infrastructure Documentation

Servers

All access credentials, including the PEM files, **for the underlisted servers are in 1Password**

Name	Public IP Address	Domains Hosted
MyAvana Live	54.147.196.34	admin.myavana.com api.myavana.com customer.myavana.com salon.myavana.com
Hair Analysis - BE	23.20.99.174	survey.myavana.com (Note that this is primarily used by Unilever)
SharedAnalysisAI_01	52.87.64.1	analysisai01.myavana.com
SharedAnalysisFE_01	54.164.3.202	integration.myavana.com policy.myavana.com (hosts the policy document we show on the retail kiosk) retail.myavana.com
Ulta	18.211.103.230	ulta.myavana.com
MyAvanaQA *Hosts all the staging services*	52.54.106.156	adminstaging.myavana.com analysisstaging.myavana.com apistaging.myavana.com customerstaging.myavana.com hairscopestaging.myavana.com integrationstaging.myavana.com proadminstaging.myavana.com proapistaging.myavana.com

Databases

All access credentials for the underlisted databases are in 1Password.

Name	Endpoint Address	Type and Usage

analytics-db	analytics-db.crws9atqqwz6.us-east-1.rds.amazonaws.com	Postgres Used for analytics
myavana	myavana.crws9atqqwz6.us-east-1.rds.amazonaws.com	SQL Server Use as **prod** instance to host all production databases for MyAdmin, Salon, Customer and Retail Kiosk
myavana-staging	myavana-staging.crws9atqqwz6.us-east-1.rds.amazonaws.com	SQL Server Use as **staging** instance to host all production databases for MyAdmin, Salon, Customer and Retail Kiosk

DNS records setting in cloudfare for authenticating our domain to send emails using Twilio sendgrid

Old send grid dns records in cloudfare.

em429 → u45352734.wl235.sendgrid.net

gor._domainkey → gor.domainkey.u45352734.wl235.sendgrid.net

gor2._domainkey → gor2.domainkey.u45352734.wl235.sendgrid.net

New records
em57 → u12658744.wl245.sendgrid.net

s1._domainkey → s1.domainkey.u12658744.wl245.sendgrid.net

s2._domainkey → s2.domainkey.u12658744.wl245.sendgrid.net

General How To’s

This section contains helpful hints for some of the outliers that occur in the course of managing our infrastructure and services.

Increasing VM Disk Size

https://docs.aws.amazon.com/ebs/latest/userguide/recognize-expanded-volume-linux.html?icmpid=docs_ec2_console

Starting Service and Troubleshooting

Commands of interest:

- sudo systemctl {reload/restart/start/stop} SERVICE_NAME